Exercise 5.4.45 Let $A \in \mathbb{R}^{n\times m}$. Then A^TA and AA^T are real symmetric matrices, so each has a spectral decomposition as guaranteed by Theorem 5.4.19. For example, there is an orthogonal matrix $W \in \mathbb{R}^{m\times m}$ and a diagonal matrix $\Lambda \in \mathbb{R}^{m\times m}$ such that $A^TA = W\Lambda W^T$. How are the spectral decompositions of A^TA and AA^T related to the singular value decomposition $A = U\Sigma V^T$? □

Exercise 5.4.46 Show that if $A \in \mathbb{C}^{n\times n}$ is normal, then its eigenvectors are singular vectors, and the eigenvalues and singular values are related by $\sigma_j = |\lambda_j|$, $j = 1,\ldots,n$, if ordered by decreasing magnitude. In particular, if A is positive definite, its eigenvalues are the same as its singular values. □

Exercise 5.4.47 Prove Theorem 5.4.22 (Wintner-Murnaghan) by induction on n, using the same general strategy as in the proof of Schur's theorem. Let λ be an eigenvalue of the matrix $A \in \mathbb{R}^{n\times n}$. If λ is real, reduce the problem exactly as in Schur's Theorem. Now let us focus on the complex case. Suppose $Av = \lambda v$, where $v = x + iy$, x, $y \in \mathbb{R}^n$, $\lambda = \alpha + i\beta$, $\alpha, \beta \in \mathbb{R}$, and $\beta \neq 0$.

(a) Show that $AZ = ZB$, where $Z = \begin{bmatrix} x & y \end{bmatrix}$ and $B = \begin{bmatrix} \alpha & \beta \\ -\beta & \alpha \end{bmatrix}$. Show that the eigenvalues of B are λ and $\overline{\lambda}$.

(b) Show that the matrix Z has rank two (full rank). In other words, x and y are linearly independent. (Show that if they are not independent, then x and y must satisfy $Ax = \lambda x$ and $Ay = \lambda y$, and λ must be real.)

(c) Consider a condensed QR decomposition (Theorem 3.4.7) $Z = VR$, where $V \in \mathbb{R}^{n\times 2}$ has orthonormal columns, and $R \in \mathbb{R}^{2\times 2}$ is upper triangular. Show that $AV = VC$, where C is similar to B.

(d) Show that there exists an orthogonal matrix $U_1 \in \mathbb{R}^{n\times n}$ whose first two columns are the columns of V. Let $A_1 = U_1^TAU_1$. Show that A_1 has the form

$$A_1 = \left[\begin{array}{cc|ccc} & C & * & \cdots & * \\ & & * & \cdots & * \\ \hline 0 & 0 & & & \\ \vdots & \vdots & & \hat{A} & \\ 0 & 0 & & & \end{array}\right],$$

where $\hat{A} \in \mathbb{R}^{(n-2)\times(n-2)}$.

(e) Apply the induction hypothesis to $\hat{A}$ to complete the proof.

(f) Write out a complete and careful proof of the Wintner-Murnaghan Theorem.

Remarks: The equation $AZ = ZB$ implies that the columns of Z span an *invariant subspace* of A. We will discuss invariant subspaces systematically in Section 6.1. Part (b) shows that this subspace has dimension 2. Part (c) introduces an orthonormal

basis for the space. We note finally that the eigenvalues and the 2×2 blocks can be made to appear in any order on the main diagonal of T. □

Exercise 5.4.48 A matrix $A \in \mathbb{R}^{n\times n}$ is *skew symmetric* if $A^T = -A$. Thus a skew-symmetric matrix is just a skew-Hermitian matrix that is real. In particular, the eigenvalues of a skew-symmetric matrix are purely imaginary (Exercise 5.4.39).

(a) Show that if $A \in \mathbb{R}^{n\times n}$ is skew symmetric and n is odd, then A is singular.

(b) What does a 1×1 skew-symmetric matrix look like? What does a 2×2 skew-symmetric matrix look like?

(c) What special form does Theorem 5.4.22 take when A is skew symmetric? Be as specific as you can.

□

Exercise 5.4.49 The *trace* of a matrix $A \in \mathbb{C}^{n\times n}$, denoted $\mathrm{tr}(A)$, is defined to be the sum of the main diagonal entries: $\mathrm{tr}(A) = \sum_{k=1}^{n} a_{kk}$.

(a) Show that for $C, D \in \mathbb{C}^{n\times n}$, $\mathrm{tr}(C + D) = \mathrm{tr}(C) + \mathrm{tr}(D)$.

(b) Show that for $C \in \mathbb{C}^{n\times m}$ and $D \in \mathbb{C}^{m\times n}$, $\mathrm{tr}(CD) = \mathrm{tr}(DC)$.

(c) Show that $\|B\|_F^2 = \mathrm{tr}(B^*B) = \mathrm{tr}(BB^*)$, where $\|\cdot\|_F$ denotes the Frobenius norm (Example 2.1.22).

(d) Suppose $A \in \mathbb{C}^{n\times n}$ is normal, and consider the partition

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix},$$

where A_{11} and A_{22} are square matrices. Show that $\|A_{21}\|_F = \|A_{12}\|_F$. (*Hint:* Write the equation $A^*A = AA^*$ in partitioned form, and take the trace of the (1,1) block.)

□

Exercise 5.4.50

(a) Suppose $T \in \mathbb{C}^{n\times n}$ is normal and has the block-triangular form

$$T = \begin{bmatrix} T_{11} & T_{12} \\ 0 & T_{22} \end{bmatrix},$$

where T_{11} and T_{22} are square matrices. Show that $T_{12} = 0$, and T_{11} and T_{22} are normal.

(b) Suppose $T \in \mathbb{C}^{n\times n}$ is normal and has the block-triangular form

$$T = \begin{bmatrix} T_{11} & T_{12} & \cdots & T_{1k} \\ 0 & T_{22} & \cdots & T_{2k} \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & T_{kk} \end{bmatrix}$$

with square blocks on the main diagonal. Use induction on k and the result of part (a) to prove that T is block diagonal and the main-diagonal blocks are normal.

□

Exercise 5.4.51 We now return our attention to real matrices.

(a) Let $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \in \mathbb{R}^{2\times 2}$. Show that A is normal if and only if either A is symmetric or A has the form $A = \begin{bmatrix} a & b \\ -b & a \end{bmatrix}$.

(b) In the symmetric case A has real eigenvalues. Find the eigenvalues of

$$A = \begin{bmatrix} a & b \\ -b & a \end{bmatrix}.$$

(c) What special form does Theorem 5.4.22 take when $A \in \mathbb{R}^{n\times n}$ is normal? Make use of the result from Exercise 5.4.50.

□

Exercise 5.4.52 Every orthogonal matrix is normal, so the results of Exercise 5.4.51 are valid for all orthogonal matrices.

(a) What does a 1×1 orthogonal matrix look like?

(b) Give necessary and sufficent conditions on a and b such that the matrix $\begin{bmatrix} a & b \\ -b & a \end{bmatrix} \in \mathbb{R}^{2\times 2}$ is orthogonal.

(c) What special form does Theorem 5.4.22 take when A is orthogonal? Be as specific as you can.

□

Exercise 5.4.53

(a) Show that if A and B are similar matrices, then $\text{tr}(A) = \text{tr}(B)$. (*Hint:* Apply part (b) of Exercise 5.4.49 with $C = S^{-1}$ and $D = AS$.)

(b) Show that the trace of a matrix equals the sum of its eigenvalues.

□

Exercise 5.4.54 Prove that the determinant of a matrix equals the product of its eigenvalues.

□

Exercise 5.4.55 Use MATLAB's `help` facility to find out about the commands `schur` and `rsf2csf`. Use them to find the forms of Schur (Theorem 5.4.11) and Winter-Murnaghan (Theorem 5.4.22) for the matrix

$$A = \begin{bmatrix} 1 & 2 & 3 \\ -4 & 5 & 6 \\ -7 & -8 & 9 \end{bmatrix}.$$

□

5.5 REDUCTION TO HESSENBERG AND TRIDIAGONAL FORMS

The results of the previous section encourage us to seek algorithms that reduce a matrix to triangular form by similarity transformations, as a means of finding the eigenvalues of the matrix. On theoretical grounds we know that there is no algorithm that accomplishes this task in a finite number of steps; such an algorithm would violate Abel's classical theorem, cited in Section 5.2. It turns out, however, that there are finite algorithms, that is, direct methods, that bring a matrix very close to upper-triangular form.

Recall that an $n \times n$ matrix A is called *upper Hessenberg* if $a_{ij} = 0$ whenever $i > j + 1$. Thus an upper Hessenberg matrix has the form

$$\begin{bmatrix} * & * & * & * & * \\ * & * & * & * & * \\ & * & * & * & * \\ & & * & * & * \\ & & & * & * \end{bmatrix}.$$

In this section we will develop an algorithm that uses unitary similarity transformations to transform a matrix to upper Hessenberg form in $\frac{10}{3}n^3$ flops. This algorithm does not of itself solve the eigenvalue problem, but it is extremely important nevertheless because it reduces the problem to a form that can be manipulated inexpensively. For example, you showed in Exercise 5.3.33 that both the LU and QR decompositions of an upper Hessenberg matrix can be computed in $O(n^2)$ flops. Thus Rayleigh quotient iteration can be performed at a cost of only $O(n^2)$ flops per iteration.

The reduction to Hessenberg form is especially helpful when the matrix is Hermitian. Since unitary similarity transformations preserve the Hermitian property, the reduced matrix is not merely Hessenberg, it is tridiagonal. That is, it has the form

$$\begin{bmatrix} * & * & & & \\ * & * & * & & \\ & * & * & * & \\ & & * & * & * \\ & & & * & * \end{bmatrix}.$$

Tridiagonal matrices can be manipulated very inexpensively. Moreover, the symmetry of the matrix can be exploited to reduce the cost the reduction to tridiagonal form to a modest $\frac{4}{3}n^3$ flops.

Exercise 5.5.1 Let $A \in \mathbb{C}^{n \times n}$ be tridiagonal.

(a) Show that the cost of an LU decomposition of A (with or without pivoting) is $O(n)$ flops.

(b) Show that the cost of a QR decomposition of A is $O(n)$ flops, provided the matrix Q is not assembled explicitly.

□

Reduction of General Matrices

The algorithm that reduces a matrix to upper Hessenberg form is quite similar to the algorithm that performs a QR decomposition using reflectors. It accomplishes the task in $n-2$ steps. The first step introduces the desired zeros in the first column, the second step takes care of the second column, and so on.

Let us proceed with the first step. Partition A as

$$A = \begin{bmatrix} a_{11} & c^T \\ b & \hat{A} \end{bmatrix}.$$

Let $\hat{Q}_1$ be a (real or complex) reflector such that $\hat{Q}_1 b = [-\tau_1, 0, \cdots, 0]^T$ ($|\tau_1| = \|b\|_2$), and let

$$Q_1 = \begin{bmatrix} 1 & 0^T \\ 0 & \hat{Q}_1 \end{bmatrix}.$$

Let

$$A_{1/2} = Q_1 A = \left[\begin{array}{c|c} a_{11} & c^T \\ \hline -\tau_1 & \\ 0 & \\ \vdots & \hat{Q}_1 \hat{A} \\ 0 & \end{array}\right],$$

which has the desired zeros in the first column. This operation is just like the first step of the QR decomposition algorithm, except that it is less ambitious. Instead of transforming all but one entry to zero in the first column, it leaves two entries nonzero. The reason for the diminished ambitiousness is that now we must complete a similarity transformation by multiplying on the right by Q_1^{-1}. Letting $A_1 = Q_1 A Q_1^{-1}$ and

recalling that $Q_1^{-1} = Q_1^* = Q_1$, we have

$$A_1 = A_{1/2}Q_1 = \left[\begin{array}{c|c} a_{11} & c^T\hat{Q}_1 \\ \hline -\tau_1 & \\ 0 & \hat{Q}_1\hat{A}\hat{Q}_1 \\ \vdots & \\ 0 & \end{array}\right] = \left[\begin{array}{c|ccc} a_{11} & * & \cdots & * \\ \hline -\tau_1 & & & \\ 0 & & & \\ \vdots & & \hat{A}_1 & \\ 0 & & & \end{array}\right].$$

Because of the form of Q_1, this operation does not destroy the zeros in the first column. If we had been more ambitious, we would have failed at this point.

The second step creates zeros in the second column of A_1, that is, in the first column of $\hat{A}_1$. Thus we pick a reflector $\hat{Q}_2 \in \mathbb{C}^{(n-2)\times(n-2)}$ in just the same way as the first step, except that A is replaced by A_1. Let

$$Q_2 = \left[\begin{array}{cc|ccc} 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ \hline 0 & 0 & & & \\ \vdots & \vdots & & \hat{Q}_2 & \\ 0 & 0 & & & \end{array}\right].$$

Then

$$A_{3/2} = Q_2A_1 = \left[\begin{array}{cc|ccc} a_{11} & * & * & \cdots & * \\ \hline -\tau_1 & * & * & \cdots & * \\ \hline 0 & -\tau_2 & & & \\ \vdots & \vdots & & \hat{Q}_2\hat{A}_2 & \\ 0 & 0 & & & \end{array}\right].$$

We complete the similarity transformation by multiplying on the right by $Q_2^{-1} = Q_2$. Because the first two columns of Q_2 are equal to the first two columns of the identity matrix, this operation does not alter the first two columns of $A_{3/2}$. Thus

$$A_2 = A_{3/2}Q_2 = \left[\begin{array}{cc|ccc} a_{11} & * & * & \cdots & * \\ \hline -\tau_1 & * & * & \cdots & * \\ \hline 0 & -\tau_2 & & & \\ \vdots & \vdots & & \hat{Q}_2\hat{A}_2\hat{Q}_2 & \\ 0 & 0 & & & \end{array}\right].$$

The third step creates zeros in the third column, and so on. After $n-2$ steps the reduction is complete. The result is an upper Hessenberg matrix that is unitarily similar to A: $B = Q^*AQ$, where

$$Q = Q_1Q_2\cdots Q_{n-2} \qquad \text{and} \qquad Q^* = Q_{n-2}Q_{n-3}\cdots Q_1.$$

If A is real, then all operations are real, Q is real and orthogonal, and B is orthogonally similar to A.

A computer program to perform the reduction can be organized in exactly the same way as a QR decomposition by reflectors is. For simplicity we will restrict our attention to the real case. Most of the details do not require discussion because they were already covered in Section 3.2. The one way in which the present algorithm is significantly different from the QR decomposition is that it involves multiplication by reflectors on the right as well as the left. As you might expect, the right multiplications can be organized in essentially the same way as the left multiplications are. If we need to calculate CQ, where $Q = I - \gamma uu^T$ is a reflector, we can write $CQ = C - \gamma Cuu^T$. Thus we can compute $v = C(\gamma u)$ and then $CQ = C - vu^T$. This is just a variant of algorithm (3.2.40).

Real Householder Reduction to Upper Hessenberg Form

for $k = 1, \ldots, n-2$

$\beta \leftarrow \max\{|\, a_{ik} \,|\; |\; i = k+1, \ldots, n\}$

$\gamma_k \leftarrow 0$

if $(\beta \neq 0)$ then

% Set up the reflector $\hat{Q}_k$

$a_{k+1:n,k} \leftarrow \beta^{-1} a_{k+1:n,k}$

$\tau_k \leftarrow \sqrt{a_{k+1,k}^2 + \cdots + a_{n,k}^2}$

if $(a_{k+1,k} < 0)$ $\tau_k \leftarrow -\tau_k$

$\eta \leftarrow a_{k+1,k} + \tau_k$

$a_{k+1,k} \leftarrow 1$

$a_{k+2:n,k} \leftarrow a_{k+2:n,k}/\eta$

$\gamma_k \leftarrow \eta/\tau_k$

$\tau_k \leftarrow \tau_k \beta$

(5.5.2)

% Multiply on the left by $\hat{Q}_k$

$b_{k+1:n,1}^T \leftarrow a_{k+1:n,k}^T a_{k+1:n,k+1:n}$

$b_{k+1:n,1}^T \leftarrow -\gamma_k b_{k+1:n,1}^T$

$a_{k+1:n,k+1:n} \leftarrow a_{k+1:n,k+1:n} + a_{k+1:n,k} b_{k+1:n,1}^T$

% Multiply on the right by $\hat{Q}_k$

$b_{1:n,1} \leftarrow a_{1:n,k+1:n} a_{k+1:n,k}$

$b_{1:n,1} \leftarrow -\gamma_k b_{1:n,1}$

$a_{1:n,k+1:n} \leftarrow a_{1:n,k+1:n} + b_{1:n,1} a_{k+1:n,k}^T$

$a_{k+1,k} \leftarrow -\tau_k$

$\tau_{n-1} \leftarrow -a_{n,n-1}$

exit

This algorithm takes as input an array that contains $A \in \mathbb{R}^{n \times n}$. It returns an upper Hessenberg matrix $B = Q^T A Q$, whose nonzero elements are in their natural positions in the array. The portion of the array below the subdiagonal is not set to zero. It is used to store the vectors u_k, used to generate the reflectors $\hat{Q}_k = I - \gamma_k u_k u_k^T$,

where $\hat{Q}_k$ is the $(n-k) \times (n-k)$ reflector used at step k. The leading 1 in u_k is not stored. The scalar γ_k $(k = 1, \ldots, n-2)$ is stored in a separate array γ. Thus the information needed to construct the orthogonal transforming matrix Q is available.

There are many ways to reorganize the computations in (5.5.2). In particular, there are block variants suitable for parallel computation and efficient use of high-speed cache memory. The Hessenberg reduction codes in LAPACK [1] use blocks.

Exercise 5.5.3 Count the flops in (5.5.2). Show that the left multiplication part costs about $\frac{4}{3}n^3$ flops, the right multiplication part costs about $2n^3$ flops, and the rest of the cost of the reflector setups is $O(n^2)$. Thus the total cost is about $\frac{10}{3}n^3$ flops. Why do the right multiplications require more flops than the left multiplications do? □

Suppose we have transformed $A \in \mathbb{R}^{n\times n}$ to upper Hessenberg form $B = Q^T A Q$ using (5.5.2). Suppose further that we have found some eigenvectors of B and we would now like to find the corresponding eigenvectors of A. For each eigenvector v of B, Qv is an eigenvector of A. Since

$$Qv = Q_1 Q_2 \cdots Q_{n-2} v,$$

we can easily calculate Qv by applying $n-2$ reflectors in succession using (3.2.40) repeatedly. In fact we can process all of the eigenvectors at once. If we have m of them, we can build an $n \times m$ matrix V whose columns are the eigenvectors, then we can compute $QV = Q_1 Q_2 \cdots Q_{n-2} V$, by applying (3.2.40) $n-2$ times. The cost is about $2n^2 m$ flops for m vectors.

If we wish to generate the transforming matrix Q itself, we can do so by computing $Q = QI = Q_1 Q_2 \cdots Q_{n-2} I$, by applying (3.2.40) $n-2$ times.

The Symmetric Case

If A is Hermitian, then the matrix B produced by (5.5.2) is not merely Hessenberg, it is tridiagonal. Furthermore it is possible to exploit the symmetry of A to reduce the cost of the reduction to $\frac{4}{3}n^3$ flops, less than half the cost of the non-symmetric case. In the interest of simplicity we will restrict our attention to the real symmetric case. We begin with the matrix

$$A = \begin{bmatrix} a_{11} & b^T \\ b & \hat{A} \end{bmatrix}.$$

In the first step of the reduction we transform A to $A_1 = Q_1 A Q_1$, where

$$A_1 = \left[\begin{array}{c|c} a_{11} & b^T\hat{Q}_1 \\ \hline \hat{Q}_1 b & \hat{Q}_1 \hat{A} \hat{Q}_1 \end{array}\right] = \left[\begin{array}{c|ccc} a_{11} & -\tau_1 & 0 & \cdots & 0 \\ \hline -\tau_1 & & & & \\ 0 & & & & \\ \vdots & & \multicolumn{3}{c}{\hat{A}_1} \\ 0 & & & & \end{array}\right].$$

We save a little bit here by not performing the computation $b^T\hat{Q}_1$, which duplicates the computation $\hat{Q}_1 b$.

The bulk of the effort in this step is expended in the computation of the symmetric submatrix $\hat{A}_1 = \hat{Q}_1 \hat{A} \hat{Q}_1$. We must do this efficiently if we are to realize significant savings. If symmetry is not exploited, it costs about $4n^2$ flops to calculate $\hat{Q}_1 \hat{A}$ and another $4n^2$ flops to calculate $(\hat{Q}_1 \hat{A})\hat{Q}_1$, as you can easily verify. Thus the entire computation of $\hat{A}_1$ costs about $8n^2$ flops. It turns out that we can cut this figure in half by carefully exploiting symmetry.

$\hat{Q}_1$ is a reflector given in the form $\hat{Q}_1 = I - \gamma u u^T$. Thus

$$\begin{aligned} \hat{A}_1 &= (I - \gamma u u^T)\hat{A}(I - \gamma u u^T) \\ &= \hat{A} - \gamma \hat{A} u u^T - \gamma u u^T \hat{A} + \gamma^2 u u^T \hat{A} u u^T. \end{aligned}$$

The terms in this expression admit considerable simplification if we introduce the auxiliary vector $v = -\gamma \hat{A} u$. We have $-\gamma \hat{A} u u^T = v u^T$, $-\gamma u u^T \hat{A} = u v^T$, and $\gamma^2 u u^T \hat{A} u u^T = -\gamma u u^T v u^T$. Introducing the scalar $\alpha = -\frac{1}{2}\gamma u^T v$, we can rewrite this last term as $2\alpha u u^T$. Thus

$$\hat{A}_1 = \hat{A} + v u^T + u v^T + 2\alpha u u^T.$$

The final manipulation is to split the last term into two pieces in order to combine one piece with the term $v u^T$ and the other with $u v^T$. In other words, let $w = v + \alpha u$. Then

$$\hat{A}_1 = \hat{A} + w u^T + u w^T.$$

This equation translates into the code segment

```
for j = 2, ..., n
    [ for i = j, ..., n
        [ a_ij ← a_ij + w_i u_j + u_i w_j
```

which costs four flops per updated array entry. By symmetry we need only update the main diagonal and lower triangle. Thus the total number of flops in this segment is $4(n-1)n/2 \approx 2n^2$. This does not include the cost of calculating w. First of all, the computation $v = -\gamma \hat{A} u$ costs about $2n^2$ flops. The computation $\alpha = -\frac{1}{2}\gamma u^T v$ costs about $2n$ flops, as does the computation $w = v + \alpha u$. Thus the total flop count for the first step is about $4n^2$, as claimed.

The second step of the reduction is identical to the first step, except that it acts on the submatrix $\hat{A}_1$. In particular, (unlike the nonsymmetric reduction) it has no effect on the first row of A_1. Thus the flop count for the second step is about $4(n-1)^2$ flops. After $n-2$ steps the reduction is complete. The total flop count is approximately $4(n^2 + (n-1)^2 + (n-2)^2 + \cdots) \approx \frac{4}{3}n^3$.

Reduction of a Real Symmetric Matrix to Tridiagonal Form

$$
\begin{array}{l}
\text{for } k = 1, \ldots, n-2 \\
\quad \left[\begin{array}{l}
\beta \leftarrow \max\{|a_{ik}| \mid i = k+1, \ldots, n\} \\
\gamma_k \leftarrow 0 \\
\text{if } (\beta \neq 0) \text{ then} \\
\text{set up the reflector } \hat{Q}_k \text{ exactly as in (5.5.2)} \\
w_{k+1:n} \leftarrow 0 \\
\text{for } j = k+1, \ldots, n \\
\quad \left[\; w_{j:n} \leftarrow w_{j:n} + a_{j:n,j} a_{jk} \right. \\
\text{for } i = k+1, \ldots, n \\
\quad \left[\; w_i \leftarrow w_i + a_{i+1:n,i}^T a_{i+1:n,k} \right. \\
w_{k+1:n} \leftarrow -\gamma_k w_{k+1:n} \\
\alpha \leftarrow w_{k+1:n} a_{k+1:n,k} \\
\alpha \leftarrow -\gamma_k \alpha / 2 \\
w_{k+1:n} \leftarrow w_{k+1:n} + \alpha a_{k+1:n,k} \\
\text{for } j = k+1, \ldots, n \\
\quad \left[\; a_{j:n,j} \leftarrow a_{j:n,j} + w_{j:n} a_{jk} + a_{j:n,k} w_j \right.
\end{array} \right. \\
\tau_{n-1} \leftarrow -a_{n,n-1} \\
\text{for } i = 1, \ldots, n \\
\quad \left[\; d_i \leftarrow a_{ii} \right. \\
s_{1:n-1} \leftarrow -\tau_{1:n-1}
\end{array}
\qquad (5.5.4)
$$

This algorithm accesses only the main diagonal and lower triangle of A. It stores the main-diagonal entries of the tridiagonal matrix B in a one-dimensional array d ($d_i = b_{ii}$, $i = 1, \ldots, n$) and the off-diagonal entries in a one-dimensional array s ($s_i = b_{i+1,i} = b_{i,i+1}$, $i = 1, \ldots, n-1$). The information about the reflectors used in the similarity transformation is stored exactly as in (5.5.2).

Additional Exercises

Exercise 5.5.5 Write a Fortran subroutine that implements (5.5.2). □

Exercise 5.5.6 Write a Fortran subroutine that implements (5.5.4). □

Exercise 5.5.7 MATLAB's `hess` command transforms a matrix to upper Hessenberg form. To get just the upper Hessenberg matrix, type `B = hess(A)`. To get both the Hessenberg matrix and the transforming matrix (fully assembled), type `[Q,B] = hess(A)`.

(a) Using MATLAB, generate a random 100×100 (or larger) matrix and reduce it to upper Hessenberg form:

```
n = 100
A = randn(n);  % or A = randn(n) + i*randn(n);
[Q,B] = hess(A);
```

(b) Use Rayleigh quotient iteration to compute an eigenpair of B, starting from a random complex vector. For example,

```
q = randn(n,1) + i*randn(n,1);
q = q/norm(q);
rayquo = q'*B*q;
qq = (B - rayquo*eye(n))\q;
```

and so on. Once you have an eigenvector q of B, transform it to an eigenvector $v = Qq$ of A. Your last Rayleigh quotient is your eigenvalue.

(c) Give evidence that your Rayleigh quotient iteration converged quadratically.

(d) Check that you really do have an eigenpair of A by computing the residual norm $\| Av - \lambda v \|_2$.

□

Exercise 5.5.8 The first step of the reduction to upper Hessenberg form introduces the needed zeros into the vector b, where

$$A = \begin{bmatrix} a_{11} & c^T \\ b & \hat{A} \end{bmatrix}.$$

We used a reflector to do the task, but there are other types of transformations that we could equally well have used. Sketch an algorithm that uses Gaussian elimination with partial pivoting to introduce the zeros at each step. Don't forget that each transformation has to be a similarity transformation. Thus we get a similarity transformation to Hessenberg form that is, however, nonunitary. This is a good algorithm that has gone out of favor. The old library EISPACK includes it, but it has been left out of LAPACK. □

5.6 THE QR ALGORITHM

For many years now the most widely used algorithm for calculating the complete set of eigenvalues of a matrix has been the QR algorithm of Francis [25] and Kublanovskaya [45]. The present section is devoted to a description of the algorithm, and in the section that follows we will show how to implement the algorithm effectively. The explanation of why the algorithm works is largely postponed to Section 6.2. You can read Sections 6.1 and 6.2 right now if you prefer.

Consider a matrix $A \in \mathbb{C}^{n \times n}$ whose eigenvalues we would like to compute. For now let us assume that A is nonsingular, a restriction that we will remove later. The basic QR algorithm is very easy to describe. It starts with $A_0 = A$ and generates a sequence of matrices (A_j) by the following prescription:

$$A_{m-1} = Q_m R_m \qquad R_m Q_m = A_m. \tag{5.6.1}$$

That is, A_{m-1} is decomposed into factors Q_m and R_m such that Q_m is unitary and R_m is upper triangular with positive entries on the main diagonal. These factors are uniquely determined (Theorem 3.2.46). The factors are then multiplied back together

in the reverse order to produce A_m. You can easily verify that $A_m = Q_m^* A_{m-1} Q_m$. Thus A_m is unitarily similar to A_{m-1}.

Exercise 5.6.2

(a) Show that $A_m = Q_m^* A_{m-1} Q_m$.

(b) Show that $A_m = R_m A_{m-1} R_m^{-1}$.

Notice that part (a) is valid, regardless of whether or not A is singular. In contrast, part (b) is valid if and only if A is nonsingular. (Why?) □

Since all matrices in the sequence (A_j) are similar, they all have the same eigenvalues. In Section 6.2 we will see that the QR algorithm is just a clever implementation of a procedure known as simultaneous iteration, which is itself a natural, easily understood extension of the power method. As a consequence, the sequence (A_j) converges, under suitable conditions, to upper-triangular form

$$\begin{bmatrix} \lambda_1 & * & \cdots & * \\ & \lambda_2 & \ddots & \vdots \\ & & \ddots & * \\ & & & \lambda_n \end{bmatrix},$$

where the eigenvalues appear in order of decreasing magnitude on the main diagonal. (As we shall see, this is a gross oversimplification of what usually happens, but there is no point in discussing the details now.)

From now on it will be important to distinguish between the *QR decomposition* and the *QR algorithm*. The *QR algorithm* is an iterative procedure for finding eigenvalues. It is based on the *QR decomposition*, which is a direct procedure related to the Gram-Schmidt process. A single iteration of the QR algorithm will be called a *QR step* or *QR iteration*.

Each QR step performs a unitary similarity transformation. We noted in Section 5.4 that numerous matrix properties are preserved under such transformations, including the Hermitian property. Thus if A is Hermitian, then all iterates A_j will be Hermitian, and the sequence (A_j) will converge to diagonal form.

If A_{m-1} is real, then Q_m, R_m, and A_m are also real. Thus if A is real, the basic QR algorithm (5.6.1) will remain within the real number system.

Example 5.6.3 Let us apply the basic QR algorithm to the real symmetric matrix

$$A = \begin{bmatrix} 8 & 2 \\ 2 & 5 \end{bmatrix},$$

whose eigenvalues are easily seen to be $\lambda_1 = 9$ and $\lambda_2 = 4$. Letting $A_0 = A$, we have $A_0 = Q_1 R_1$, where

$$Q_1 = \frac{1}{\sqrt{68}} \begin{bmatrix} 8 & -2 \\ 2 & 8 \end{bmatrix} \quad \text{and} \quad R_1 = \frac{1}{\sqrt{68}} \begin{bmatrix} 68 & 26 \\ 0 & 36 \end{bmatrix}.$$

Thus

$$A_1 = R_1 Q_1 = \frac{1}{68}\begin{bmatrix} 596 & 72 \\ 72 & 288 \end{bmatrix} \approx \begin{bmatrix} 8.7647 & 1.0588 \\ 1.0588 & 4.2353 \end{bmatrix}.$$

Notice that A_1 is real and symmetric, just as A_0 is. Notice also that A_1 is closer to diagonal form than A_0 is, in the sense that its off-diagonal entries are closer to zero. Moreover, the main-diagonal entries of A_1 are closer to the eigenvalues. On subsequent iterations the main-diagonal entries of A_j give progressively better estimates of the eigenvalues, until after ten iterations they agree with the true eigenvalues to seven decimal places. □

Exercise 5.6.4 Let $A_0 = A = \begin{bmatrix} 8 & 1 \\ -2 & 1 \end{bmatrix}$. Find an orthogonal Q_1 (a rotator) and an upper-triangular R_1 such that such that $A_0 = Q_1 R_1$. Calculate $A_1 = R_1 Q_1$. Notice that A_1 is closer to upper-triangular form than A_0 is, and the main-diagonal entries of A_1 are closer to the eigenvalues of A than those of A_0 are. □

The assumption that A is nonsingular guarantees that every matrix in the sequence (A_j) is nonsingular. This fact allows us to specify the decomposition $A_{m-1} = Q_m R_m$ uniquely by requiring that the main-diagonal entries of R_m be positive. Thus the QR algorithm, as described above, is well defined. However, it is not always convenient in practice to arrange the computations so that each R_m has positive main-diagonal entries. It is therefore reasonable to ask how the sequence (A_j) is affected if this requirement is dropped. Exercise 5.6.24 shows that nothing bad happens. Whether the requirement is enforced or not makes no significant difference in the progress of the algorithm. Therefore, in actual implementations of the QR algorithm we will not require that the main-diagonal entries of each R_m be positive.

There are two reasons why the basic QR algorithm (5.6.1) is too inefficient for general use. First, the cost of each QR step is high. Each QR decomposition costs $\frac{4}{3}n^3$ flops, and the matrix multiplication that follows also costs $O(n^3)$ flops. This is a very high price to pay, given that we expect to have to perform quite a few steps. The second problem is that convergence is generally quite slow; a very large number of iterations is needed before A_m is sufficiently close to triangular form that we are willing to accept its main-diagonal entries as eigenvalues of A. Thus we need to make QR steps less expensive, and we need to accelerate the convergence somehow, so that fewer QR iterations are needed.

QR Algorithm with Hessenberg Matrices

The cost of QR iterations can be reduced drastically by first reducing the matrix to Hessenberg form. This works because the upper Hessenberg form is preserved by the QR algorithm, as the following theorem shows.

Theorem 5.6.5 *Let A_{m-1} be a nonsingular upper Hessenberg matrix, and suppose A_m is obtained from A_{m-1} by one QR iteration (5.6.1). Then A_m is also in upper Hessenberg form.*

Proof. The equation $A_{m-1} = Q_m R_m$ can be rewritten as $Q_m = A_{m-1} R_m^{-1}$. By Exercise 1.7.44, R_m^{-1} is upper triangular. You can easily show (Exercise 5.6.6) that the product of an upper Hessenberg matrix with an upper triangular matrix, in either order, is upper Hessenberg. Therefore Q_m is an upper Hessenberg matrix. But then $A_m = R_m Q_m$ must also be upper Hessenberg. □

Exercise 5.6.6 Suppose $H \in \mathbb{C}^{n \times n}$ is upper Hessenberg and $R \in \mathbb{C}^{n \times n}$ is upper triangular. Prove that both RH and HR are upper Hessenberg. □

In Theorem 5.6.5 we assumed once again that the matrix is nonsingular. It would be nice to drop this requirement. Unfortunately Theorem 5.6.5 is not strictly true in the singular case. The problem is that in that case there is extra freedom (i.e. more serious loss of uniqueness) in the construction of the QR decomposition, which makes it possible to build a Q_m and an R_m for which the resulting A_m is not upper Hessenberg (see Exercise 5.6.25). Fortunately this is not a serious problem. As we shall now show, it is always possible to construct the Q and R factors in such a way that Hessenberg form is preserved. In fact, if one carries out the QR decomposition in the most straightforward, obvious way, that's what happens.

Suppose A is an upper Hessenberg matrix on which we wish to carry out a QR step. We can transform A to upper triangular form by using $n-1$ rotators to transform the $n-1$ subdiagonal entries to zero. For what follows you might find it useful to refer back to the material on rotators in Section 3.2. If you find it helpful to think only in terms of the real case, then do so by all means.

We begin by finding a (complex) rotator Q_1 acting in the 1–2 plane such that $Q_1^* A$ has a zero in the $(2,1)$ position (Exercise 3.2.54). This rotator alters only the first and second rows of the matrix. Next we find a rotator Q_2 acting in the 2–3 plane such that $Q_2^* Q_1^* A$ has a zero in the $(3,2)$ position. Q_2^* alters only the second and third rows of $Q_1^* A$. Since the intersection of these rows with the first column consists of zeros, these zeros will not be destroyed by the transformation; they will be recombined to create new zeros. In particular, the zero in the $(2,1)$ position, which was created by the previous rotator, is preserved. Next we find Q_3, a rotator in the 3–4 plane, such that $Q_3^* Q_2^* Q_1^* A$ has a zero in position $(4,3)$. You can easily check that this rotator does not destroy the zeros that were created previously or any other zeros below the main diagonal.

Continuing in this manner, we transform A to an upper triangular matrix R given by

$$R = Q_{n-1}^* \cdots Q_2^* Q_1^* A.$$

Letting

$$Q = Q_1 Q_2 \cdots Q_{n-1}, \tag{5.6.7}$$

we have $R = Q^* A$ or $A = QR$, where Q is unitary and R is upper triangular.

Now we must calculate $A_1 = RQ$ to complete a step of the QR algorithm. By (5.6.7)

$$A_1 = R Q_1 Q_2 \cdots Q_{n-1},$$

so all we need to do is multiply R on the right by the rotators $Q_1, Q_2, \ldots, Q_{n-1}$ successively. Since Q_1 acts in the 1–2 plane, it recombines the first and second columns of the matrix. Since both of these columns consist of zeros after the first two positions, these zeros will not be destroyed by Q_1. The only zero that can (and almost certainly will) be destroyed is the one in the $(2,1)$ position. Similarly Q_2, which acts on columns 2 and 3, can destroy only the zero in the $(3,2)$ position, and so on. Thus the transformation from R to A_1 can create new nonzero entries below the main diagonal only in positions $(2,1), (3,2), \ldots, (n, n-1)$. We conclude that A_1 is an upper Hessenberg matrix.

Exercise 5.6.8 Show that the matrix Q defined by (5.6.7) is upper Hessenberg. (Start with the identity matrix and build up Q by applying the rotators one at a time.) This, together with Exercise 5.6.6, gives a second way of seeing that $A_1 = RQ$ is upper Hessenberg. □

The construction that we have just worked through produces an A_1 that is in upper Hessenberg form, regardless of whether A is nonsingular or not. Thus, for all practical purposes, we can say that the QR algorithm preserves upper Hessenberg form. This observation will be strengthened by Exercise 5.6.28 below. By the way, the construction can also be carried out using (2×2) reflectors in place of plane rotators. Whichever we use, the construction also yields the following result.

Theorem 5.6.9 *A QR step applied to an upper Hessenberg matrix requires not more than $O(n^2)$ flops.*

Proof. The construction outlined above transforms A_{m-1} to A_m by applying $n-1$ rotators on the left followed by $n-1$ rotators on the right. The cost of applying each of these rotators (or reflectors) is $O(n)$ flops. Thus the total flop count is $O(n^2)$. The exact count depends on the details of the implementation. □

We have now fairly well solved the problem of the cost of QR steps. To summarize, we begin by reducing the matrix to upper Hessenberg form at a cost of $O(n^3)$ flops. While this is expensive, it only has to be done once, because upper Hessenberg form is preserved by the QR algorithm. The QR iterations are then relatively inexpensive, each one costing only $O(n^2)$ flops.

The situation is even better when the matrix is Hermitian, since the Hermitian Hessenberg form is tridiagonal. The fact that the QR algorithm preserves both the Hermitian property and the upper Hessenberg form implies that the tridiagonal Hermitian form is preserved by the QR steps. Such QR steps are extremely inexpensive.

Exercise 5.6.10 Show that a QR iteration applied to a Hermitian tridiagonal matrix requires only $O(n)$ flops. □

Accelerating the Convergence of the QR Algorithm

Consider a sequence (A_j) of iterates of the QR algorithm, where every A_m is in upper Hessenberg form, and let $a_{ij}^{(m)}$ denote the (i,j) entry of A_m. Thus

$$A_m = \begin{bmatrix} a_{11}^{(m)} & a_{12}^{(m)} & \cdots & a_{1,n-1}^{(m)} & a_{1n}^{(m)} \\ a_{21}^{(m)} & a_{22}^{(m)} & \cdots & a_{2,n-1}^{(m)} & a_{2n}^{(m)} \\ & a_{32}^{(m)} & \cdots & a_{3,n-1}^{(m)} & a_{3n}^{(m)} \\ & & \ddots & \vdots & \vdots \\ & & & a_{n,n-1}^{(m)} & a_{nn}^{(m)} \end{bmatrix}.$$

Let $\lambda_1, \lambda_2, \ldots, \lambda_n$ denote the eigenvalues of A, ordered so that $|\lambda_1| \geq |\lambda_2| \geq \cdots \geq |\lambda_n|$. In Section 6.2 we will see that (most of) the subdiagonal entries $a_{i+1,i}^{(m)}$ converge to zero as $m \to \infty$. More precisely, if $|\lambda_i| > |\lambda_{i+1}|$, then $a_{i+1,i}^{(m)} \to 0$ linearly, with convergence ratio $|\lambda_{i+1}/\lambda_i|$, as $m \to \infty$. Thus we can improve the rate of convergence by decreasing one or more of the ratios $|\lambda_{i+1}/\lambda_i|$, $i = 1, \ldots, n-1$. An obvious way to do this is to shift the matrix.

The shifted matrix $A - \rho I$ has eigenvalues $\lambda_1 - \rho, \lambda_2 - \rho, \ldots, \lambda_n - \rho$. if we renumber the eigenvalues so that $|\lambda_1 - \rho| \geq |\lambda_2 - \rho| \geq \cdots \geq |\lambda_n - \rho|$, then the ratios associated with $A - \rho I$ are $|(\lambda_{i+1} - \rho)/(\lambda_i - \rho)|$, $i = 1, \ldots, n-1$. The one ratio that can be made really small is $|(\lambda_n - \rho)/(\lambda_{n-1} - \rho)|$, which we can make as close to zero as we please (provided $\lambda_n \neq \lambda_{n-1}$) by choosing ρ very close to λ_n.

We do not mean to imply here that ρ has to approximate the particular eigenvalue that is named λ_n. If we can find a ρ that approximates any one of the eigenvalues well, then that eigenvalue will be called λ_n after the renumbering has been done. Thus if we can find a ρ that is an excellent approximation of any one of the eigenvalues, we can gain by applying the QR algorithm to $A - \rho I$ instead of A. The entry $a_{n,n-1}^{(m)}$ will converge to zero very quickly. Once it is sufficiently small, it can be considered to be zero for practical purposes, and adding the shift back on, we have

$$A_m + \rho I = \left[\begin{array}{ccc|c} & & & * \\ & \hat{A}_m & & \vdots \\ & & & * \\ \hline 0 & \cdots & 0 & a_{nn}^{(m)} \end{array}\right].$$

By Theorem 5.2.10 $a_{nn}^{(m)}$ is an eigenvalue of A; indeed $a_{nn}^{(m)} = \lambda_n$, the eigenvalue closest to ρ. The remaining eigenvalues of A are eigenvalues of $\hat{A}_m$, so we might as well economize by performing subsequent iterations on this smaller matrix. If we can find a $\hat{\rho}$ that approximates an eigenvalue of $\hat{A}_m$ well, then we can extract that eigenvalue quickly by performing QR iterations on $\hat{A}_m - \hat{\rho} I$. Once that eigenvalue has been found, we can go after the next eigenvalue, operating on an even smaller matrix, and so on. Continuing in this fashion, operating on smaller and smaller matrices, we eventually find all of the eigenvalues of A. This process, through which the size of the matrix is reduced each time an eigenvalue is found, is called *deflation*.

The catch to this argument is that we need good approximations to the eigenvalues. Where can we obtain these approximations? Suppose we begin by performing several QR iterations with no shift. After a number of iterations the matrices will begin to approach triangular form, and the main-diagonal entries will begin to approach the eigenvalues. In particular, $a_{nn}^{(m)}$ will approximate λ_n, the eigenvalue of A of least modulus. It is therefore reasonable to take $\rho = a_{nn}^{(m)}$ at some point and perform subsequent iterations on the shifted matrix $A_m - \rho I$. In fact we can do better than that. With each step we get a better approximation to λ_n. There is no reason why we should not update the shift frequently in order to improve the convergence rate. In fact we can choose a new shift at each step if we want to. This is exactly what is done in the *shifted QR algorithm*:

$$A_{m-1} - \rho_{m-1} I = Q_m R_m \qquad R_m Q_m + \rho_{m-1} I = A_m, \tag{5.6.11}$$

where at each step ρ_{m-1} is chosen to approximate whichever eigenvalue is emerging at the lower right-hand corner of the matrix.

Exercise 5.6.12 Show that if A_{m-1} and A_m are related as in (5.6.11), then $A_m = Q_m^* A_{m-1} Q_m$. Thus A_{m-1} and A_m are unitarily similar. □

We have tentatively decided that the choice $\rho_{m-1} = a_{nn}^{(m-1)}$ is good. This is called the *Rayleigh quotient shifting strategy* or, simply, *Rayleigh quotient shift*, because $a_{nn}^{(m-1)}$ can be viewed as a Rayleigh quotient. See Exercise 5.6.26.

Since the shifts ultimately converge to λ_n, the convergence ratios

$$|(\lambda_n - \rho_m)/(\lambda_{n-1} - \rho_m)|$$

tend to zero, provided $\lambda_n \neq \lambda_{n-1}$. Therefore the convergence is faster than linear. In Section 6.2 we will show that the QR algorithm with the Rayleigh quotient shift carries out Rayleigh quotient iteration implicitly as a part of its action. From this it follows that the convergence is quadratic. If the matrix is Hermitian (or, more generally, normal) the convergence is cubic.

How many unshifted QR steps do we need to take before $a_{nn}^{(m)}$ can be accepted as a good approximation to λ_n? Experience has shown that there is no need to wait until $a_{nn}^{(m)}$ approximates λ_n well; it is generally safe to start shifting right from the very first step. The only effect this has is that the initial shifts alter the order of the eigenvalues so that they do not necessarily emerge in order of increasing magnitude.

Example 5.6.13 Consider again the matrix

$$A = \begin{bmatrix} 8 & 2 \\ 2 & 5 \end{bmatrix}$$

of Example 5.6.3, whose eigenvalues are $\lambda_1 = 9$ and $\lambda_2 = 4$. When the unshifted QR algorithm is applied to A, the $(2,1)$ entry converges to zero linearly with convergence ratio $|\lambda_2/\lambda_1| = 4/9$. Now let us try the shifted QR algorithm with the Rayleigh

quotient shift. Thus we take $\rho_0 = 5$ and perform a QR step with $A - \rho_0 I$, whose eigenvalues are $\lambda_1 - 5 = 4$ and $\lambda_2 - 5 = -1$. The ratio $|(\lambda_2 - \rho_0)/(\lambda_1 - \rho_0)| = 1/4$ is less than 4/9, so we expect to do better with the shifted step. $A_0 - \rho_0 I = Q_1 R_1$, where

$$Q_1 = \frac{1}{\sqrt{13}} \begin{bmatrix} 3 & 2 \\ 2 & -3 \end{bmatrix} \quad \text{and} \quad R_1 = \frac{1}{\sqrt{13}} \begin{bmatrix} 13 & 6 \\ 0 & 4 \end{bmatrix}.$$

Thus

$$\begin{aligned} A_1 = R_1 Q_1 + \rho_0 I &= \frac{1}{13} \begin{bmatrix} 51 & 8 \\ 8 & -12 \end{bmatrix} \\ &\approx \begin{bmatrix} 8.9231 & 0.6154 \\ 0.6154 & 4.0769 \end{bmatrix}. \end{aligned}$$

Comparing this result with that of Example 5.6.3, we see that with one shifted QR step we have made more progress toward convergence than we did with an unshifted step. Not only is the off-diagonal element smaller; the main-diagonal entries are now quite close to the eigenvalues. The Rayleigh quotient shift for the next step is $\rho_1 \approx 4.0769$. The eigenvalues of $A_1 - \rho_1 I$ are $\lambda_1 - \rho_1 = 4.9231$ and $\lambda_2 - \rho_1 = -0.0769$, whose ratio is 0.0156. We therefore expect that A_2 will be substantially better that A_1. Indeed this is the case. Using MATLAB we find that

$$A_2 = \begin{bmatrix} 8.99998092658643 & 0.00976558774724 \\ 0.00976558774724 & 4.00001907341357 \end{bmatrix}.$$

On the next step the shift is $\rho_2 = 4.00001907341357$, which gives an even better ratio, and

$$A_3 = \begin{bmatrix} 9.00000000000000 & 0.00000003725290 \\ 0.00000003725290 & 4.00000000000000 \end{bmatrix}.$$

Finally

$$A_4 = \begin{bmatrix} 9.00000000000000 & 0.00000000000000 \\ 0.00000000000000 & 4.00000000000000 \end{bmatrix}.$$

□

Exercise 5.6.14 It is easy to write a MATLAB program to execute the QR algorithm using MATLAB's built-in QR decomposition command; for example, you can use commands like

```
shift = A(n,n);
[Q,R] = qr(A-shift*eye(n));
A = R*Q + shift*eye(n)
```

This is an inefficient implementation, but it is adequate for the purpose of experimentation with small matrices. Let

$$A = \begin{bmatrix} 8 & 1 \\ -2 & 1 \end{bmatrix},$$

as in Exercise 5.6.4.

(a) Using MATLAB, perform four or more steps of the QR algorithm with zero shift. Show that $a_{21}^{(m)} \to 0$ and $a_{11}^{(m)} \to \lambda_1$ linearly with convergence ratio approximately $|\lambda_2/\lambda_1|$.

(b) Using MATLAB, perform four or more steps of the QR algorithm with the Rayleigh quotient shift. Give evidence that $a_{21}^{(m)} \to 0$ and $a_{11}^{(m)} \to \lambda_1$ quadratically.

(c) Using MATLAB, perform one step of the QR algorithm with shift $\rho = 1.29843788128358$ (an eigenvalue).

□

Exercise 5.6.15 The point of this exercise is to show how inefficient the sample code from the previous exercise is. Run the following MATLAB code, adjusting the matrix size to fit the speed of your computer.

```
n = 300;
A = randn(n);
tic; [Q,R] = qr(A); B = R*Q; toc
tic; [V,D] = eig(A); toc
```

The `eig` command reduces the matrix to upper Hessenberg form and then performs approximately $2n$ implicit double QR iterations (described in Section 5.7) to obtain the eigenvalues. It then computes a complete set of eigenvectors using methods like those described in Section 5.8. It does all this in a modest multiple of the time it takes to do one QR iteration using the inefficient code. □

Example 5.6.16 This example shows that the Rayleigh quotient shifting strategy does not always work. Consider the real, symmetric matrix

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix},$$

whose eigenvalues are easily seen to be $\lambda_1 = 3$ and $\lambda_2 = 1$. The Rayleigh quotient shift is $\rho = 2$, which lies half way between the eigenvalues. The shifted matrix $A - \rho I$ has eigenvalues ± 1, which have the same magnitude. Since $A - \rho I$ is unitary, its QR factors are $Q_1 = A - \rho I$ and $R_1 = I$. Thus $A_1 = R_1 Q_1 + \rho I = A$. Thus the QR step leaves A fixed. The algorithm "cannot decide" which eigenvalue to approach. We have already observed this phenomenon in connection with Rayleigh quotient iteration (Exercise 5.3.35). □

Because the Rayleigh quotient iteration fails occasionally, a different shift, the *Wilkinson shift*, is preferred. The *Wilkinson shift* is defined to be that eigenvalue of the trailing 2×2 submatrix

$$\begin{bmatrix} a_{n-1,n-1}^{(m-1)} & a_{n-1,n}^{(m-1)} \\ a_{n,n-1}^{(m-1)} & a_{n,n}^{(m-1)} \end{bmatrix} \tag{5.6.17}$$

that is closer to $a_{n,n}^{(m-1)}$. It is not difficult to calculate this shift, since the eigenvalues of a 2×2 matrix can be found by the quadratic formula. Because the Wilkinson shift uses a greater amount of information from A_{m-1}, it is not unreasonable to expect that it would give a better approximation to the eigenvalue. In the case of real, symmetric tridiagonal matrices this expectation is confirmed by a theorem that states that the QR algorithm with the Wilkinson shift always converges. The rate of convergence is usually cubic or better. For details see [54], which also discusses a number of other shifting strategies for symmetric matrices. For general matrices there still remain some very special cases for which the Wilkinson shift fails. An example will be given below.

For the vast majority of matrices the Wilkinson shift strategy works very well. Experience has shown that typically only about five to nine QR steps are needed before the first eigenvalue emerges. While $a_{n,n-1}^{(m)}$ converges to zero rapidly, the other subdiagonal entries of A_m move slowly toward zero. As a consequence, by the time the first eigenvalue has emerged, some progress toward convergence of the other eigenvalues has been made. This slow progress continues as subsequent eigenvalues are extracted. Therefore, on average, the later eigenvalues will emerge in fewer iterations than the first ones did. It commonly happens that many of the later eigenvalues emerge after two or fewer steps. The average is in the range of three to five iterations per eigenvalue. For Hermitian matrices the situation is better; only some two to three iterations are needed per eigenvalue.

It sometimes happens during the course of QR iterations that one of the subdiagonal entries other than the bottom one becomes (practically) zero. In really large matrices, this is a common event. Whenever it happens, the problem can be *reduced*; that is, it can be broken into two smaller problems. Suppose, for example, $a_{i+1,i}^{(m)} = 0$. Then A_m has the form

$$A_m = \begin{bmatrix} B_{11} & B_{12} \\ 0 & B_{22} \end{bmatrix},$$

where $B_{11} \in \mathbb{C}^{j \times j}$, $B_{22} \in \mathbb{C}^{k \times k}$, $j + k = n$. Now we can find the eigenvalues of B_{11} and B_{22} separately. This can save a lot of work if the break is near the middle of the matrix. Since the cost of a Hessenberg QR iteration is $O(n^2)$, if we cut n in half, we divide the cost of a QR iteration by a factor of four.

An upper Hessenberg matrix whose subdiagonal entries are all nonzero is called an *unreduced* or *proper* upper Hessenberg matrix. I prefer the latter term because the former is illogical. Because every upper Hessenberg matrix that has zeros on the subdiagonal can be broken into submatrices, it is never necessary to perform a QR step on a matrix that is not properly upper Hessenberg. This fact is crucial to the development of the implicit QR algorithm, which will be discussed in the next section.

Complex Eigenvalues of Real Matrices

Most eigenvalue problems that arise in practice involve real matrices. The fact that real matrices can have complex eigenvalues exposes another weakness of the

Rayleigh quotient shift. The Rayleigh quotient shift associated with a real matrix is necessarily real, so it cannot approximate a non-real eigenvalue well. The Wilkinson shift, in contrast, can be nonreal, for it is an eigenvalue of a 2×2 matrix. Thus this shifting strategy allows the possibility of approximating complex eigenvalues of real matrices well.

This brings us to an important point. When working with real matrices, we would prefer to stay within the real number system as much as possible. the use of complex shifts would appear to force us into the complex number field. Fortunately we can avoid complex numbers by implementing the QR algorithm with some care. First of all, complex eigenvalues of real matrices occur in conjugate pairs. Thus, as soon as we know one eigenvalue λ, we immediately know another, $\overline{\lambda}$. In the interest of staying within the real number system, we might try to design an algorithm that seeks these two eigenvalues simultaneously. Such an algorithm would not extract them one at a time, deflating twice, rather it would extract a real 2×2 block whose eigenvalues are λ and $\overline{\lambda}$. How might we build such an algorithm?

The Wilkinson shift strategy computes the eigenvalues of the trailing 2×2 submatrix (5.6.17). Assuming A_{m-1} is real, if one eigenvalue of (5.6.17) is complex, then so is the other; they are complex conjugates ρ_{m-1} and $\overline{\rho}_{m-1}$. Since $A_{m-1} - \rho_{m-1} I$ is complex, a QR step with shift ρ_{m-1} will result in a complex matrix A_m. However it can (and will) be shown that if this step is followed immediately by a QR step with shift $\overline{\rho}_{m-1}$, then the resulting A_{m+1} is real again.

In the next section we will develop the double-step QR algorithm, which constructs A_{m+1} directly from A_{m-1}, bypassing A_m. The computation is carried out entirely in real arithmetic and costs about as much as two ordinary (real) QR steps. The iterates satisfy $a^{(m)}_{n-1,n-2} \to 0$ instead of $a^{(m)}_{n,n-1} \to 0$. The convergence is normally quadratic, so after a fairly small number of iterations $a^{(m)}_{n-1,n-2}$ is small enough to be considered zero for practical purposes. Then

$$A_m = \left[\begin{array}{ccc|cc} & & & * & * \\ & \hat{A}_m & & \vdots & \vdots \\ & & & * & * \\ \hline 0 & \cdots & 0 & \multicolumn{2}{c}{\multirow{2}{*}{Λ_m}} \\ 0 & \cdots & 0 & & \end{array}\right],$$

where $\Lambda_m \in \mathbb{R}^{2\times 2}$ has eigenvalues λ and $\overline{\lambda}$, which can be computed (carefully) by the quadratic formula (Exercise 5.6.27). The remaining eigenvalues of A_m are all eigenvalues of $\hat{A}_m$, so subsequent iterations can operate on the submatrix $\hat{A}_m$. Thus a double deflation takes place.

Exceptional Shifts

There are some very special situations in which the Wilkinson shift fails. Consider the following example.

Example 5.6.18 Let

$$A = \begin{bmatrix} & & & & 1 \\ 1 & & & & \\ & 1 & & & \\ & & \ddots & & \\ & & & 1 & \end{bmatrix}. \tag{5.6.19}$$

The subdiagonal consists entirely of 1's, and there is a 1 in the $(1, n)$ position. All other entries are zero. This properly upper Hessenberg matrix is unitary, since its columns form an orthonormal set. Therefore, in the QR decomposition of A, $Q = A$ and $R = I$. Thus $A_1 = RQ = A$; the QR step goes nowhere. Clearly this will happen whenever A is unitary. This does not contradict the convergence results cited earlier. All eigenvalues of a unitary matrix lie on the unit circle of the complex plane, so the ratios $|\lambda_{j+1}/\lambda_j|$ are all 1. This symmetric situation will be broken up by any nonzero shift. Notice, however that for the matrix (5.6.19) both the Rayleigh quotient shift and the Wilkinson shift are zero. Thus both of these shifting strategies fail. □

Because of the existence of matrices like the one in Example 5.6.18, the standard QR programs for the nonsymmetric eigenvalue problem include an *exceptional shift* feature. Whenever it appears that the algorithm may not be converging, as indicated by many steps having passed since the last deflation, one step with an exceptional shift is taken. The point of the exceptional shift is to break up any unfortunate symmetries that might be impeding convergence. The exact value of the shift is unimportant; it could be a random number for example, although it should be of the same magnitude as elements of the matrix. Exceptional shifts are needed only rarely.

A question that remains open is to find a shifting strategy that normally gives rapid (e.g. quadratic) convergence and is guaranteed to lead to convergence in every case.

The Singular Case

In our development of the QR algorithm, singular matrices have required special attention. Since matrices that are exactly singular hardly ever arise in practice, it is tempting to save some effort by ignoring them completely. However, to do so would be to risk leaving you with a misconception about the algorithm. If every result were prefaced by the words, "Assume A is nonsingular," you might get the impression that the singular case is a nasty one that we hope to avoid at all costs. In fact, just the opposite is true. The goal of the shifting strategies is to find shifts that are as close to eigenvalues as possible. The closer ρ is to an eigenvalue, the closer the shifted matrix $A - \rho I$ is to having a zero eigenvalue, that is, to being singular. If a shift that is exactly an eigenvalue should be chosen, $A - \rho I$ would be singular. From this point of view the singular case appears to be very desirable and worthy of special attention. This is confirmed by the next theorem, which shows that if the QR algorithm is applied to a properly upper Hessenberg matrix, using an exact eigenvalue as a shift, then, in principal, just one QR step suffices to extract the eigenvalue. In preparation for the theorem, you should work the following exercise.

Exercise 5.6.20 Prove the following two statements.

(a) If $A \in \mathbb{C}^{n\times n}$ is properly upper Hessenberg, then the first $n-1$ columns of A are linearly independent.

(b) If $R \in \mathbb{C}^{n\times n}$ is an upper-triangular matrix whose first $n-1$ columns are linearly independent, then the first $n-1$ main-diagonal entries, $r_{11}, r_{22}, \ldots, r_{n-1,n-1}$ must all be nonzero.

□

Theorem 5.6.21 *Let $A \in \mathbb{C}^{n\times n}$ be a singular, properly upper Hessenberg matrix, and let B be the result of one step of the QR algorithm with shift 0, starting from A. Then the entire last row of B consists of zeros. In particular, $b_{nn} = 0$ is an eigenvalue and can be removed immediately by deflation.*

Exercise 5.6.22 Read the first paragraph of the following proof, then complete the proof yourself using the results from Exercise 5.6.20. □

Proof. $B = RQ$, where $A = QR$, Q is unitary, and R is upper triangular. Since A is singular and Q is nonsingular, R must be singular. This implies that at least one of the main-diagonal entries r_{ii} is zero.

From Exercise 5.6.20, part (a), we know that the first $n-1$ columns of A are linearly independent. Let $a_1, a_2, \ldots, a_{n-1}$ denote these columns, and let $r_1, r_2, \ldots, r_{n-1}$ denote the first $n-1$ columns of R. From the equation $R = Q^*A$ we see that $r_1 = Q^*a_1$, $r_2 = Q^*a_2, \ldots, r_{n-1} = Q^*a_{n-1}$. Since Q^* is nonsingular and $a_1, a_2, \ldots, a_{n-1}$ are linearly independent, $r_1, r_2, \ldots, r_{n-1}$ must also be linearly independent. Therefore, by Exercise 5.6.20, part (b), the first $n-1$ main-diagonal entries of R are nonzero. We have already observed that at least one of the main-diagonal entries of R must be zero, so $r_{nn} = 0$. Since R is upper triangular, this means that the entire last row of R consists of zeros. Since $B = RQ$, the entire last row of B must also consist of zeros. □

Corollary 5.6.23 *Let λ be an eigenvalue of the properly upper Hessenberg matrix $A \in \mathbb{C}^{n\times n}$. Let B be the result of one QR step of the QR algorithm with shift λ. Then the last row of B is $[0, \cdots, 0, \lambda]$. Thus the eigenvalue λ can be removed immediately by deflation.*

Theorem 5.6.21 and Corollary 5.6.23 are true in exact arithmetic. In practice, roundoff errors will cause $b_{n,n-1}$ to be not quite zero. In most cases $b_{n,n-1}$ will be far enough from zero to prevent deflation. When this happens, one additional QR step (with the same shift) is usually enough to allow deflation.

Additional Exercises

Exercise 5.6.24 This exercise shows that in the QR algorithm the requirement that the main diagonal entries of the R factors be positive is inessential.

(a) Suppose A is nonsingular and $A = QR = \tilde{Q}\tilde{R}$, where Q and $\tilde{Q}$ are unitary and R and $\tilde{R}$ are upper triangular. Since we have placed no restriction on the main-diagonal entries of R or $\tilde{R}$, it can happen that $Q \neq \tilde{Q}$ and $R \neq \tilde{R}$. Show that there is a unitary diagonal matrix D such that $\tilde{Q} = QD$ and $\tilde{R} = D^*R$. (*Hint:* Rewrite the equation $QR = \tilde{Q}\tilde{R}$ so that there is a unitary matrix on one side and an upper-triangular matrix on the other. Then use the fact that a matrix that is both unitary and upper triangular must be diagonal (Exercise 5.4.40).)

(b) Suppose A is nonsingular. Let $A_0 = A$ and $\tilde{A}_0 = A$, and let (A_j) and $(\tilde{A}_j)$ be sequences that satisfy

$$A_{m-1} = Q_m R_m \qquad R_m Q_m = A_m,$$

$$\tilde{A}_{m-1} = \tilde{Q}_m \tilde{R}_m \qquad \tilde{R}_m \tilde{Q}_m = \tilde{A}_m,$$

where Q_m and $\tilde{Q}_m$ are unitary and R_m and $\tilde{R}_m$ are upper triangular. Prove by induction on m that there is a sequence of unitary diagonal matrices (D_m) such that $\tilde{A}_m = D_m^* A_{m-1} D_m$ for all m.

(c) Let $a_{ij}^{(m)}$ and $\tilde{a}_{ij}^{(m)}$ denote the (i,j) entry of A_m and $\tilde{A}_m$, respectively. show that $|a_{ij}^{(m)}| = |\tilde{a}_{ij}^{(m)}|$ for all i and j, and $a_{ii}^{(m)} = \tilde{a}_{ii}^{(m)}$ for all i.

□

Exercise 5.6.25 Let

$$A_0 = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix},$$

which is obviously upper Hessenberg and singular. There is no need to apply the QR algorithm to this matrix, but we will do so anyway, just to make a point.

(a) Let

$$Q_1 = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \quad \text{and} \quad R_1 = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}.$$

Show that $A_0 = Q_1 R_1$, and that this is a QR decomposition of A_0. Let $A_1 = R_1 Q_1$. Show that A_1 is not upper Hessenberg.

(b) Produce a different QR decomposition of A_0 for which the resulting A_1 is upper Hessenberg.

(c) Show where the proof of Theorem 5.6.5 breaks down in the singular case.

□

Exercise 5.6.26 Let $A = (a_{ij}) \in \mathbb{C}^{n \times n}$.

(a) Show that $a_{nn} = e_n^* A e_n$, where e_n denotes the nth standard basis vector. Thus the shift $\rho = a_{nn}$ is a Rayleigh quotient.

(b) Let A be upper Hessenberg. Give an informal argument that indicates that e_n is approximately an eigenvector of A^T if $a_{n,n-1}$ is small, the approximation improving as $a_{n,n-1} \to 0$.

Thus the Rayleigh quotient shift is the Rayleigh quotient associated with an approximate eigenvector. □

Exercise 5.6.27 This exercise discusses the careful calculation of the eigenvalues of a real 2×2 matrix. Suppose we want to find the eigenvalues of

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \in \mathbb{R}^{2\times 2}.$$

A straightforward application of the quadratic formula to the characteristic polynomial can sometimes give inaccurate results. We can avoid difficulties by performing an orthogonal similarity transformation to get A into a special form. Let

$$Q = \begin{bmatrix} c & s \\ -s & c \end{bmatrix}$$

be a rotator. Thus $c = \cos\theta$ and $s = \sin\theta$ for some θ. Let $\hat{A} = Q^T A Q$.

(a) It turns out to be desirable to choose Q so that $\hat{A}$ satisfies $\hat{a}_{11} = \hat{a}_{22}$. Show that if $\hat{a}_{11} = \hat{a}_{22}$, then the eigenvalues of A and $\hat{A}$ are

$$\hat{a}_{11} \pm \sqrt{\hat{a}_{21}\hat{a}_{12}}.$$

Show that the eigenvalues are a complex conjugate pair if $\hat{a}_{21}$ and $\hat{a}_{12}$ have opposite sign. This formula computes the complex eigenvalues with no cancellations, so it computes eigenvalues that are as accurate as the data warrants. A transformation of this type is used by the routines in LAPACK. In part (b) you will show that such a transformation is always possible. The case of real eigenvalues is dealt with further in part (c).

(b) Show that

$$\hat{A} = \begin{bmatrix} c^2a_{11} + s^2a_{22} - cs(a_{12} + a_{21}) & c^2a_{12} - s^2a_{21} + cs(a_{11} - a_{22}) \\ c^2a_{21} - s^2a_{12} + cs(a_{11} - a_{22}) & c^2a_{22} + s^2a_{11} + cs(a_{12} + a_{21}) \end{bmatrix}.$$

Using the trigonometric identities $\cos 2\theta = c^2 - s^2$ and $\sin 2\theta = 2sc$, show that $\hat{a}_{11} = \hat{a}_{22}$ if and only if

$$\cos 2\theta(a_{11} - a_{22}) = \sin 2\theta(a_{12} + a_{21}).$$

Show that if $a_{12} + a_{21} = 0$, we achieve $\hat{a}_{11} = \hat{a}_{22}$ by taking $c = s = 1/\sqrt{2}$. Otherwise, choose θ so that

$$\hat{t} = \tan 2\theta = \frac{a_{11} - a_{22}}{a_{12} + a_{21}}$$

to get the desired result. Let $t = \tan\theta = s/c$. Use half-angle formulas and other trigonometric identities to show that

$$t = \frac{\sin 2\theta}{1 + \cos 2\theta} = \frac{\hat{t}}{1 + \sqrt{1 + \hat{t}}}.$$

Show further that $c = 1/\sqrt{1 + t^2}$ and $s = ct$. An alternative formula for t that use $\cot 2\theta$ instead of $\tan 2\theta$ (useful when $|a_{11} - a_{22}| \gg |a_{12} + a_{21}|$) can be inferred from Exercise 6.6.46.

(c) If the eigenvalues turn out to be real, that is, if $\hat{a}_{21}\hat{a}_{12} \geq 0$ a second rotator can be applied to transform $\hat{A}$ to upper triangular form. To keep the notation simple, let us drop the hat from $\hat{A}$. That is, we assume A already satisfies $a_{11} = a_{22}$ and $a_{21}a_{12} \geq 0$, and we seek a transformation $\hat{A} = Q^T A Q$ for which $\hat{a}_{21} = 0$. Show that the condition $a_{11} = a_{22}$ implies that $\hat{A}$ has the simpler form

$$\hat{A} = \begin{bmatrix} a_{11} - cs(a_{12} + a_{21}) & c^2 a_{12} - s^2 a_{21} \\ c^2 a_{21} - s^2 a_{12} & a_{11} + cs(a_{12} + a_{21}) \end{bmatrix}.$$

Show that if $a_{12} = 0$, we obtain the desired result by taking $c = 0$ and $s = 1$. Otherwise, show that $\sqrt{a_{21}/a_{12}}$ is a real number, and $t = s/c = \sqrt{a_{21}/a_{12}}$ gives the desired transformation. Then $\hat{a}_{11}$ and $\hat{a}_{22}$ are the eigenvalues of A.

□

Exercise 5.6.28 Let $A \in \mathbb{C}^{n\times n}$ be a properly upper Hessenberg matrix with QR decomposition $A = QR$.

(a) Let $\hat{A}, \hat{Q} \in \mathbb{C}^{n\times(n-1)}$ consist of the first $n-1$ columns of A and Q, respectively. Show that there is a nonsingular, upper-triangular matrix $\hat{R} \in \mathbb{C}^{(n-1)\times(n-1)}$ such that $\hat{Q} = \hat{A}\hat{R}^{-1}$. (Use ideas from the proof of Theorem 5.6.21.)

(b) Use the result of part (a) to show that Q is an upper Hessenberg matrix.

(c) Conclude that B is upper Hessenberg, where $B = RQ$.

Thus the QR algorithm applied to a properly upper Hessenberg matrix always preserves the upper Hessenberg form, regardless of whether or not the matrix is singular. This result clearly applies to the shifted QR algorithm as well. □

Exercise 5.6.29 Let $A \in \mathbb{C}^{n\times n}$ be the matrix of Example 5.6.18.

(a) Let $\omega \in \mathbb{C}$ be an eigenvalue of A with associated eigenvector v. Write down and examine the equation $Av = \omega v$. Demonstrate that v must be a multiple of the vector

$$\begin{bmatrix} \omega^{n-1} \\ \vdots \\ \omega^2 \\ \omega \\ 1 \end{bmatrix}, \tag{5.6.30}$$

and ω must be an nth root of unity. (The *nth roots of unity* are the complex numbers that satisfy $\omega^n = 1$. There are n of them, and they are spaced evenly around the unit circle.) Conversely, show that if ω is an nth root of unity, then ω is an eigenvalue of A with eigenvector v given by (5.6.30). Thus the eigenvalues of A are exactly the nth roots of unity.

(b) It follows from part (a) that the characteristic polynomial of A is $\lambda^n - 1$ (Why?). Prove this fact independently using the formula $\det(\lambda I - A)$ and induction on n.

Notice that A is a companion matrix (5.2.15), and compare this exercise with Exercise 5.2.23. □

5.7 IMPLEMENTATION OF THE QR ALGORITHM

In this section we introduce the implicit QR algorithm and discuss practical implementation details for both the single-step and double-step versions. Our approach is nonstandard [50], [80]. The usual approach, which invokes the so-called Implicit-Q Theorem (Theorems 5.7.23 and 5.7.24), has the advantage of brevity. The advantage of our approach is that it shows more clearly the relationship between the explicit and implicit versions of the QR algorithm.

The algorithms discussed here apply a sequence of rotators or reflectors to the matrix under consideration. Therefore they are normwise backward stable (See Section 3.2). This means that each eigenvalue that they compute is the true eigenvalue of a matrix $A + \delta A$, where $\|\delta A\|/\|A\|$ is tiny. This does not guarantee that the computed eigenvalue is close to a true eigenvalue of A unless the eigenvalue is well conditioned. Condition numbers and sensitivity of eigenvalues are discussed in Section 6.5.

We shall assume throughout this section that our matrices are real. The extension to the complex case is routine.

Implicit QR algorithm

A shifted QR step has the form

$$A - \rho I = QR, \qquad RQ + \rho I = \hat{A}. \tag{5.7.1}$$

The resulting matrix $\hat{A}$ is similar to A by the orthogonal similarity transformation

$$\hat{A} = Q^T A Q. \tag{5.7.2}$$

Notice the notation $\hat{A}$ in place of A_1. In this section we will use the symbol A_i to denote the ith partial result in a single QR step rather than the ith iteration of the QR algorithm.

We shall assume that A and $\hat{A}$ are in upper Hessenberg form, and A is properly upper Hessenberg. If the step (5.7.1) is programmed in a straightforward manner,

with Q formed as a product of rotators or reflectors, a very satisfactory algorithm results. It is called the *explicit* QR algorithm. It has been found, however, that on rare occasions the operation of subtracting ρI from A causes vital information to be lost through cancellation. The *implicit* QR algorithm is based on a different way of carrying out the step. Instead of performing (5.7.1), the implicit QR algorithm carries out the similarity transformation (5.7.2) directly by applying a sequence of rotators or reflectors to A. Since the shift is never actually subtracted from A, this algorithm is more stable. The cost of an implicit QR step turns out to be almost exactly the same as that of an explicit step, so the implicit algorithm is generally preferred. Furthermore, the promised double-step QR algorithm is based on the same ideas.

To see how to perform a QR step implicitly, let us take a close look at how the explicit algorithm works. As explained in the previous section, $A - \rho I$ can be reduced to upper triangular form by a sequence of plane rotators:[7]

$$R = Q_{n-1}^T \cdots Q_2^T Q_1^T (A - \rho I). \tag{5.7.3}$$

The rotator Q_i^T acts on rows i and $i+1$ and annihilates the entry $a_{i+1,i}$. The transforming matrix Q in (5.7.2) is given by $Q = Q_1 Q_2 \cdots Q_{n-1}$. Our approach will be to generate the rotators Q_i one by one and perform the similarity transformation (5.7.2) by stages: Starting with $A_0 = A$, we generate intermediate results $A_1, A_2, A_3, \ldots$ by

$$A_i = Q_i^T A_{i-1} Q_i, \qquad i = 1, \ldots, n-1, \tag{5.7.4}$$

and end with $A_{n-1} = \hat{A}$. The trick is to generate the rotators Q_i without actually performing the decomposition (5.7.3) or even forming the matrix $A - \rho I$.

We begin with Q_1. The job of Q_1^T is to annihilate a_{21} in $A - \rho I$. That is, Q_1^T acts on the first two rows of $A - \rho I$ and effects the transformation

$$\begin{bmatrix} a_{11} - \rho \\ a_{21} \end{bmatrix} \to \begin{bmatrix} * \\ 0 \end{bmatrix}.$$

Let $\alpha = \sqrt{(a_{11} - \rho)^2 + a_{21}^2}$. Since $a_{21} \neq 0$, we have $\alpha \neq 0$. Thus we can define

$$c = (a_{11} - \rho)/\alpha \qquad \text{and} \qquad s = a_{21}/\alpha. \tag{5.7.5}$$

Then the rotator

$$Q_1^T = \left[\begin{array}{cc|c} c & s & \\ -s & c & \\ \hline & & I \end{array}\right] \tag{5.7.6}$$

does the job. We have the numbers a_{11}, a_{21}, and ρ on hand, and these are all that are needed to calculate Q_1. In particular, there is no need to transform A to $A - \rho I$.

Once we have Q_1, we calculate $A_1 = Q_1^T A Q_1$. Let us see what A_1 looks like. Multiplication of A on the left by Q_1^T alters only the first two rows. This

[7]For simplicity we speak of rotators. In every case a reflector would work just as well.

transformation preserves the upper Hessenberg form of the matrix. (It does *not* annihilate a_{21} unless ρ happens to be zero.) Multiplication of $Q_1^T A$ by Q_1 on the right recombines the first two columns. The resulting matrix has the form

$$\left[\begin{array}{cc|ccccc} a_{11}^{(1)} & a_{12}^{(1)} & a_{13}^{(1)} & \cdot & \cdot & a_{1,n}^{(1)} \\ a_{21}^{(1)} & a_{22}^{(1)} & a_{23}^{(1)} & \cdot & \cdot & a_{2,n}^{(1)} \\ \hline a_{31}^{(1)} & a_{32}^{(1)} & a_{33} & \cdot & \cdot & a_{3,n} \\ 0 & 0 & a_{43} & \cdot & \cdot & a_{4,n} \\ \vdots & \vdots & \vdots & \ddots & & \vdots \\ 0 & 0 & 0 & \cdots & a_{n,n-1} & a_{n,n} \end{array}\right],$$

where superscripts mark entries that have been altered. This matrix fails to be upper Hessenberg because the $(3,1)$ entry is nonzero. We call this entry the *bulge*.

If we were doing an explicit QR step, we would have computed

$$R_1 = Q_1^T(A - \rho I) = \left[\begin{array}{c|ccccc} * & * & * & \cdots & * & * \\ \hline 0 & x & * & \cdots & * & * \\ 0 & y & a_{33} - \rho & \cdots & & a_{3n} \\ 0 & 0 & a_{43} & \cdots & & a_{4n} \\ \vdots & \vdots & \ddots & & & \vdots \\ 0 & 0 & 0 & & a_{n,n-1} & a_{nn} - \rho \end{array}\right].$$

We would then take Q_2^T to be the rotator acting on rows two and three that effects the transformation

$$\begin{bmatrix} x \\ y \end{bmatrix} \rightarrow \begin{bmatrix} * \\ 0 \end{bmatrix}.$$

Thus we need to know x and y (more precisely, we need to know their proportions) in order to calculate Q_2.

Since we are not doing an explicit QR step, we have A_1, not R_1. Our challenge is now to determine Q_2 from A_1. Fortunately it turns out that the vector $\begin{bmatrix} a_{21}^{(1)} \\ a_{31}^{(1)} \end{bmatrix}$, taken from the bulge region of A_1, is proportional to $\begin{bmatrix} x \\ y \end{bmatrix}$. That is,

$$\begin{bmatrix} a_{21}^{(1)} \\ a_{31}^{(1)} \end{bmatrix} = \beta \begin{bmatrix} x \\ y \end{bmatrix}$$

for some nonzero β. We shall defer the proof until later. Thus the knowledge of $a_{21}^{(1)}$ and $a_{31}^{(1)}$ allows us to calculate Q_2. Indeed, letting $\gamma = \sqrt{(a_{21}^{(1)})^2 + (a_{31}^{(1)})^2}$, $c = a_{21}^{(1)}/\gamma$, and $s = a_{31}^{(1)}/\gamma$, we have

$$Q_2^T = \left[\begin{array}{c|cc|c} 1 & & & \\ \hline & c & s & \\ & -s & c & \\ \hline & & & I \end{array}\right].$$

The nice thing about Q_2^T is that when we apply it to A_1, it annihilates the bulge: $Q_2^T A_1$ is upper Hessenberg. When we apply Q_2 on the right to complete the computation of A_2, the second and third columns are recombined, and we obtain

$$A_2 = \begin{bmatrix} a_{11}^{(2)} & a_{12}^{(2)} & a_{13}^{(2)} & \cdots & & \\ a_{21}^{(2)} & a_{22}^{(2)} & a_{23}^{(2)} & \cdots & & \\ 0 & a_{32}^{(2)} & a_{33}^{(2)} & \cdots & & \\ 0 & a_{42}^{(2)} & a_{43}^{(2)} & \cdots & & \\ \vdots & & & \ddots & & \\ & & & & a_{n,n-1} & a_{n,n} \end{bmatrix},$$

which would be upper Hessenberg, except that it has a new bulge in the $(4,2)$ position. One can say that the transformation $A_1 \rightarrow A_2$ chased the bulge from position $(3,1)$ to $(4,2)$.

The reader can easily guess how the algorithm proceeds from here. To get A_3 we need Q_3^T, which is a rotator acting on rows three and four. It turns out that Q_3^T is exactly the rotator that effects the transformation

$$\begin{bmatrix} a_{32}^{(2)} \\ a_{42}^{(2)} \end{bmatrix} \rightarrow \begin{bmatrix} * \\ 0 \end{bmatrix}.$$

Thus $Q_3^T A_2$ is again upper Hessenberg, and $A_3 = Q_3^T A_2 Q_3$ has a new bulge in the $(5,3)$ position.

Each subsequent rotator chases the bulge down and over one position. The matrix Q_{n-1}^T acts on the bottom two rows, annihilating the bulge in position $(n, n-2)$. Multiplication by Q_{n-1} on the right does not create a new one. The implicit QR step is complete. For obvious reasons the implicit QR algorithm is called a *bulge-chasing* algorithm.

We have yet to prove that the rotators that chase the bulge are the right rotators for a QR step. To this end it is convenient to introduce some more notation. For $i = 1, \ldots, n-1$, let $\hat{Q}_i = Q_1 Q_2 \cdots Q_i$ and (cf. (5.7.3))

$$R_i = Q_i^T \cdots Q_2^T Q_1^T (A - \rho I) = \hat{Q}_i^T (A - \rho I). \tag{5.7.7}$$

Then $R = R_{n-1}$. For each i, Q_{i+1}^T is the rotator that transforms R_i to R_{i+1}. In particular, it effects the transformation

$$\begin{bmatrix} r_{i+1,i+1}^{(i)} \\ r_{i+2,i+1}^{(i)} \end{bmatrix} \rightarrow \begin{bmatrix} r_{i+1,i+1}^{(i+1)} \\ 0 \end{bmatrix}.$$

We know that A_1 is upper Hessenberg, except for a bulge in position $(3,1)$. Now assume inductively that A_i is upper Hessenberg, except for a bulge in position $(i+2, i)$. We need to show that

$$\begin{bmatrix} a_{i+1,i}^{(i)} \\ a_{i+2,i}^{(i)} \end{bmatrix} = \delta \begin{bmatrix} r_{i+1,i+1}^{(i)} \\ r_{i+2,i+1}^{(i)} \end{bmatrix}$$

for some nonzero δ. This will prove that Q_{i+1}^T is exactly the transformation that annihilates the bulge in A_i. It follows that A_{i+1} is almost upper Hessenberg, having a bulge in position $(i+3, i+1)$.

The desired conclusion follows from the equation

$$A_i R_i = R_i A, \tag{5.7.8}$$

which is proved as follows. Since $A_i = \hat{Q}_i^T A \hat{Q}_i$, we have $A_i \hat{Q}_i^T = \hat{Q}_i^T A$. Multiplying this equation on the right by $(A - \rho I)$, using the definition $R_i = \hat{Q}_i^T (A - \rho I)$ and the obvious fact $A(A - \rho I) = (A - \rho I)A$, we obtain (5.7.8).

Now rewrite (5.7.8) in partitioned form as

$$\begin{bmatrix} A_{11}^{(i)} & A_{12}^{(i)} \\ A_{21}^{(i)} & A_{22}^{(i)} \end{bmatrix} \begin{bmatrix} R_{11}^{(i)} & R_{12}^{(i)} \\ 0 & R_{22}^{(i)} \end{bmatrix} = \begin{bmatrix} R_{11}^{(i)} & R_{12}^{(i)} \\ 0 & R_{22}^{(i)} \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \tag{5.7.9}$$

where the $(1,1)$ blocks are $i \times i$. $R_{11}^{(i)}$ is upper triangular, and $R_{22}^{(i)}$ is upper Hessenberg. Calculating the $(2,1)$ block of the product on each side, we find that

$$A_{21}^{(i)} R_{11}^{(i)} = R_{22}^{(i)} A_{21}. \tag{5.7.10}$$

There are many zeros in this equation. Since A is upper Hessenberg, A_{21} has only one nonzero entry, $a_{i+1,i}$, in the upper right-hand corner. Similarly, by the induction hypothesis A_i is almost Hessenberg, so $A_{21}^{(i)}$ has only two nonzero entries, $a_{i+1,i}^{(i)}$ and $a_{i+2,i}^{(i)}$. In particular, only the last column of (5.7.10) is nonzero. Equating last columns in (5.7.10), we find that

$$\begin{bmatrix} a_{i+1,i}^{(i)} \\ a_{i+2,i}^{(i)} \end{bmatrix} r_{ii}^{(i)} = \begin{bmatrix} r_{i+1,i+1}^{(i)} \\ r_{i+2,i+1}^{(i)} \end{bmatrix} a_{i+1,i}. \tag{5.7.11}$$

Since A is a proper Hessenberg matrix, both $a_{i+1,i}$ and $r_{ii}^{(i)}$ are nonzero. Thus we have obtained the desired result.

Exercise 5.7.12 Write a pseudocode algorithm that performs an implicit QR step. □

Exercise 5.7.13 The *proper* Hessenberg property is crucial to establishing the proportionality of the vectors in (5.7.11). What happens to an implicit QR step if $a_{i+1,i} = 0$ for some i? □

Exercise 5.7.14 Consider the following algorithm.

(i) Form $A_1 = Q_1^T A Q_1$ (creating a bulge)
(ii) Reduce A_1 to upper Hessenberg form by an orthogonal similarity transformation as in Section 5.5.

Show that this algorithm performs an implicit QR step. □

Symmetric Matrices

The implicit QR step is simplified considerably when A is symmetric (or Hermitian in the complex case). In this case both A and $\hat{A}$ are symmetric and tridiagonal. The intermediate matrices A_1, A_2, A_3, ... are also symmetric. Each fails to be tridiagonal only in that it has a bulge, which appears both below the subdiagonal and above the superdiagonal. A good implementation of the implicit QR step will take full advantage of the symmetry. For example, in the transformation from A to A_1, it is inefficient to calculate the unsymmetric intermediate result $Q_1^T A$. It is better to make the transformation from A to $Q_1^T A Q_1$ directly, exploiting the symmetry of both matrices to minimize the computational effort.

In a computer program the matrices can be stored very compactly. A one-dimensional array of length n suffices to store the main diagonal, an array of length $n-1$ can store the subdiagonal (= superdiagonal), and a single additional storage location can hold the bulge. Each intermediate matrix can be stored over the previous one, so only one such set of arrays is needed. The imposition of this data structure not only saves space, it has the added virtue of forcing the programmer to exploit the symmetry and streamline the computations. The details are worked out in Exercises 5.7.29 through 5.7.34.

The Double-Step QR Algorithm

Let $A \in \mathbb{R}^{n\times n}$ be a real, properly upper Hessenberg matrix, and consider a pair of QR steps with shifts ρ and τ, which may be complex:

$$\begin{array}{ll} A - \rho I = Q_\rho R_\rho & R_\rho Q_\rho + \rho I = \breve{A} \\ \breve{A} - \tau I = Q_\tau R_\tau & R_\tau Q_\tau + \tau I = \hat{A} \end{array} \tag{5.7.15}$$

Since $\breve{A} = Q_\rho^* A Q_\rho$ and $\hat{A} = Q_\tau^* \breve{A} Q_\tau$, we have $\hat{A} = Q^* A Q$, where $Q = Q_\rho Q_\tau$. If ρ and τ are both real, then all intermediate matrices in (5.7.15) will be real, as will $\hat{A}$. If complex shifts are used, complex matrices will result. However, if ρ and τ are chosen so that $\tau = \overline{\rho}$, then $\hat{A}$ turns out to be real, even though the intermediate matrices are complex. The following lemma is the crucial result.

Lemma 5.7.16 *Let $Q = Q_\rho Q_\tau$ and $R = R_\tau R_\rho$, where Q_ρ, Q_τ, R_ρ, and R_τ are given by (5.7.15). Then*

$$(A - \tau I)(A - \rho I) = QR$$

Proof. Since $Q_\rho^* A Q_\rho = \breve{A}$, we easily deduce that $(A - \tau I)Q_\rho = Q_\rho(\breve{A} - \tau I)$. Therefore $(A-\tau I)(A-\rho I) = (A-\tau I)Q_\rho R_\rho = Q_\rho(\breve{A}-\tau I)R_\rho = Q_\rho Q_\tau R_\tau R_\rho = QR$. □

Lemma 5.7.17 *Suppose $\tau = \overline{\rho}$ in (5.7.15). Then*

(a) *$(A - \tau I)(A - \rho I)$ is real.*

(b) *If ρ and τ are not eigenvalues of A, then the matrices Q and R of Lemma 5.7.16 are real, and the matrix $\hat{A}$ generated by (5.7.15) is also real.*

Proof. (a) $(A-\overline{\rho}I)(A-\rho I) = A^2-(\overline{\rho}+\rho)A+\overline{\rho}\rho I$. The coefficients $\rho+\overline{\rho} = 2\Re(\rho)$ and $\overline{\rho}\rho = |\rho|^2$ are both real, so $(A-\overline{\rho}I)(A-\rho I)$ is real.
(b) Since $A-\rho I$ and $A-\tau I$ are both nonsingular, the QR decompositions in (5.7.15) are unique, provided R_ρ and R_τ are taken to have real, positive entries on the main diagonal. Then $Q = Q_\rho Q_\tau$ is unitary and $R = R_\tau R_\rho$ is upper triangular with real, positive entries on the main diagonal. Thus, by Lemma 5.7.16, Q and R are the unique factors in the QR decomposition of the matrix $(A-\tau I)(A-\rho I)$. But this matrix is real, so its Q and R factors must also be real. Thus $\hat{A} = Q^*AQ = Q^TAQ$ is real. □

Once again we have made the assumption of nonsingularity. The singular case is troublesome because then Q and R are not uniquely determined. It turns out that there is nothing to worry about: Q can always be chosen so that it is real. The resulting $\hat{A}$ is consequently also real. The algorithm we are about to develop always produces a real $\hat{A}$, regardless of whether or not the shifts are eigenvalues.

Our goal is to make the transformation from the real matrix A directly to the real matrix $\hat{A}$, doing all calculations in real arithmetic. We will begin with an impractical solution to our problem. Then we will see how to make the algorithm practical.

Here is the impractical approach. First construct the real matrix

$$B = (A-\overline{\rho}I)(A-\rho I) = A^2 - 2\Re(\rho)A + |\rho|^2 I. \tag{5.7.18}$$

Then calculate the real QR decomposition $B = QR$. Finally, perform the real, orthogonal similarity transformation $\hat{A} = Q^TAQ$. Let us call this an *explicit* double QR step.

Exercise 5.7.19 Use MATLAB to perform an explicit double QR steps on the matrix

$$\begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 0 & 9 & 10 & 11 \\ 0 & 0 & -12 & 13 \end{bmatrix}.$$

Use the eigenvalues of the lower-right-hand 2×2 submatrix as shifts. Iterate until you have isolated a pair of eigenvalues. Some sample code:

```
A = [1 2 3 4; 5 6 7 8; 0 9 10 11; 0 0 -12 13]
n = 4
format long
shift = eig(A(n-1:n,n-1:n))
B = real((A-shift(1)*eye(n))*(A-shift(2)*eye(n)));
[Q,R] = qr(B);
A = Q'*A*Q
```

What evidence for convergence is there? How many iterations does it take to get a pair of eigenvalues? What rate of convergence do you observe? □

This procedure is unsatisfactory for several reasons. For one thing, the computation of B requires that A be multiplied by itself. This costs $O(n^3)$ flops, even though

A is upper Hessenberg. If n is large, we cannot afford to spend n^3 flops on each iteration. But let us imagine, for the sake of argument, that we have the matrix B in hand. What does B look like?

Exercise 5.7.20

(a) Show that the matrix B of (5.7.18) satisfies $b_{ij} = 0$ if $i > j + 2$. (We call such a matrix a *2-Hessenberg* matrix.)

(b) Show that $b_{ij} \neq 0$ if $i = j + 2$. Thus B is *properly* 2-Hessenberg.

□

Since B is almost upper triangular, its QR decomposition can be computed cheaply. In the first column of B, only the first three entries are nonzero. Thus this column can be transformed to upper triangular form by a reflector that acts only on the first three rows (or by a pair of plane rotators). Call this reflector Q_1^T $(= Q_1)$. Similarly, since the nonzero part of the second column of B extends only to b_{42}, it can be converted to upper triangular form by a reflector Q_2^T that acts only on rows two through four. Continuing in this manner, we obtain upper triangular form after applying $n - 1$ such reflectors:

$$R = Q_{n-1}^T \cdots Q_2^T Q_1^T B.$$

Each reflector acts on three rows, except Q_{n-1}^T, which acts on only two rows. We have $Q = Q_1 Q_2 \cdots Q_{n-1}$, but obviously we should not multiply the Q_i together. Rather, we should perform the transformation $\hat{A} = Q^T A Q$ by stages: Let $A_0 = A$ and

$$A_i = Q_i^T A_{i-1} Q_i, \qquad i = 1, \ldots, n-1. \tag{5.7.21}$$

Then $\hat{A} = A_{n-1}$. Each of the transformations can be done in $O(n)$ flops, since each reflector acts on only three or fewer rows or columns. Thus the entire transformation from A to $\hat{A}$ can be accomplished in $O(n^2)$ flops, if we have the Q_i.

Now the only question is how to determine the Q_i without ever computing $B = (A - \tau I)(A - \rho I)$. Let us start with Q_1. This requires only the first column of B, which is much cheaper to compute than the entire matrix. The first column has only three nonzero entries, and these are easily seen to be

$$\begin{aligned} b_{11} &= a_{21}\left[\frac{a_{11}^2 - (\rho+\tau)a_{11} + \rho\tau}{a_{21}} + a_{12}\right] \\ b_{21} &= a_{21}\left[(a_{11} + a_{22}) - (\rho + \tau)\right] \\ b_{31} &= a_{21}a_{32}. \end{aligned} \tag{5.7.22}$$

These simple computations give us the information we need to determine Q_1, which is the reflector acting on rows 1–3 that effects the transformation

$$\begin{bmatrix} b_{11} \\ b_{21} \\ b_{31} \end{bmatrix} \rightarrow \begin{bmatrix} r_{11} \\ 0 \\ 0 \end{bmatrix}.$$

Since any multiple of this vector is equally good for the determination of Q_1, the nonzero common factor a_{21} is normally divided out.

Once we have Q_1, we can use it to calculate $A_1 = Q_1^T A Q_1$, which is not an upper Hessenberg matrix. Since this transformation recombines rows 1–3 and columns 1–3, A_1 has the form

$$A_1 = \begin{bmatrix} a_{11}^{(1)} & a_{12}^{(1)} & a_{13}^{(1)} & a_{14}^{(1)} & a_{15}^{(1)} & a_{16}^{(1)} & \cdots \\ a_{21}^{(1)} & a_{22}^{(1)} & a_{23}^{(1)} & a_{24}^{(1)} & a_{25}^{(1)} & a_{26}^{(1)} & \\ a_{31}^{(1)} & a_{32}^{(1)} & a_{33}^{(1)} & a_{34}^{(1)} & a_{35}^{(1)} & a_{36}^{(1)} & \\ a_{41}^{(1)} & a_{42}^{(1)} & a_{43}^{(1)} & a_{44} & a_{45} & a_{46} & \\ 0 & 0 & 0 & a_{54} & a_{55} & a_{56} & \\ 0 & 0 & 0 & 0 & a_{65} & a_{66} & \\ \vdots & & & & & & \ddots \end{bmatrix},$$

which would be upper Hessenberg, except for a bulge consisting of the three entries $a_{31}^{(1)}$, $a_{42}^{(1)}$, and $a_{41}^{(1)}$.

Now we need to determine Q_2. The explicit algorithm would require that we calculate $R_1 = Q_1^T B$. Then Q_2 is the reflector acting on rows 2–4 that effects the transformation

$$\begin{bmatrix} r_{22}^{(1)} \\ r_{32}^{(1)} \\ r_{42}^{(1)} \end{bmatrix} \rightarrow \begin{bmatrix} * \\ 0 \\ 0 \end{bmatrix}.$$

We shall prove below (Exercise 5.7.37) that the vectors

$$\begin{bmatrix} r_{22}^{(1)} \\ r_{32}^{(1)} \\ r_{42}^{(1)} \end{bmatrix} \quad \text{and} \quad \begin{bmatrix} a_{21}^{(1)} \\ a_{31}^{(1)} \\ a_{41}^{(1)} \end{bmatrix}$$

are proportional. Thus Q_2 can be determined from A_1 as the reflector that effects the transformation

$$\begin{bmatrix} a_{21}^{(1)} \\ a_{31}^{(1)} \\ a_{41}^{(1)} \end{bmatrix} \rightarrow \begin{bmatrix} * \\ 0 \\ 0 \end{bmatrix}.$$

Once we have Q_2, we can compute $A_2 = Q_2^T A_1 Q_2$. The transformation $A_1 \rightarrow Q_2^T A_1$ returns column 1 to upper Hessenberg form. The transformation $Q_2^T A_1 \rightarrow A_2$ then recombines columns 2–4, creating new nonzeros in positions $(5, 2)$ and $(5, 3)$.

Thus

$$A_2 = \begin{bmatrix} a_{11}^{(2)} & a_{12}^{(2)} & a_{13}^{(2)} & a_{14}^{(2)} & a_{15}^{(2)} & a_{16}^{(2)} & \cdots \\ a_{21}^{(2)} & a_{22}^{(2)} & a_{23}^{(2)} & a_{24}^{(2)} & a_{25}^{(2)} & a_{26}^{(2)} & \\ 0 & a_{32}^{(2)} & a_{33}^{(2)} & a_{34}^{(2)} & a_{35}^{(2)} & a_{36}^{(2)} & \\ 0 & a_{42}^{(2)} & a_{43}^{(2)} & a_{44}^{(2)} & a_{45}^{(2)} & a_{46}^{(2)} & \\ 0 & a_{52}^{(2)} & a_{53}^{(2)} & a_{54}^{(2)} & a_{55} & a_{56} & \\ 0 & 0 & 0 & 0 & a_{65} & a_{66} & \\ \vdots & & & & & & \ddots \end{bmatrix}.$$

The bulge has moved down and to the right. This establishes the pattern for the process. As we shall see (Exercise 5.7.37), Q_3 is the reflector acting on rows 3–5 that effects the transformation

$$\begin{bmatrix} a_{32}^{(2)} \\ a_{42}^{(2)} \\ a_{52}^{(2)} \end{bmatrix} \rightarrow \begin{bmatrix} * \\ 0 \\ 0 \end{bmatrix}.$$

Thus the transformation $A_3 \rightarrow A_4$ pushes the bulge one more column over and down. Each subsequent transformation chases the bulge further until it finally disappears off the bottom of the matrix. With $\hat{A} = A_{n-1}$, we have completed an implicit double QR step.

The implicit double QR step is justified in Exercise 5.7.37. Implementation details are discussed in Exercises 5.7.40 through 5.7.44.

The Implicit-Q Theorem

For future reference, we state here the implicit-Q theorem, which is used in the usual approach to justifying the implicit QR algorithm. We begin with the strict version of the theorem.

Theorem 5.7.23 *Let A, $\hat{A}$, $\tilde{A}$, $\hat{Q}$, and $\tilde{Q} \in \mathbb{C}^{n\times n}$. Suppose $\hat{A}$ and $\tilde{A}$ are proper upper Hessenberg matrices with positive entries on the subdiagonal, $\hat{Q}$ and $\tilde{Q}$ are unitary,*

$$\hat{A} = \hat{Q}^{-1}A\hat{Q}, \qquad \text{and} \qquad \tilde{A} = \tilde{Q}^{-1}A\tilde{Q}.$$

Suppose further that the first columns of $\hat{Q}$ and $\tilde{Q}$ are equal. Then

$$\hat{Q} = \tilde{Q} \qquad \text{and} \qquad \hat{A} = \tilde{A}.$$

In other words, a unitary similarity transformation to proper upper Hessenberg form is uniquely determined by its first column. This holds if we insist that the subdiagonal entries of the resulting proper Hessenberg matrix are all positive. In practice, we usually do not bother to make those entries positive, so the following, relaxed version of the theorem is of interest.

Theorem 5.7.24 *Let A, $\hat{A}$, $\tilde{A}$, $\hat{Q}$, and $\tilde{Q} \in \mathbb{C}^{n\times n}$. Suppose $\hat{A}$ properly upper Hessenberg, $\tilde{A}$ is upper Hessenberg, $\hat{Q}$ and $\tilde{Q}$ are unitary,*

$$\hat{A} = \hat{Q}^{-1}A\hat{Q}, \qquad \text{and} \qquad \tilde{A} = \tilde{Q}^{-1}A\tilde{Q}.$$

Suppose further that the first columns of $\hat{Q}$ and $\tilde{Q}$ are proportional; that is, $\hat{q}_1 = \tilde{q}_1 d_1$, where $|d_1| = 1$. Then $\tilde{A}$ is also properly upper Hessenberg, and there is a unitary diagonal matrix D such that

$$\hat{Q} = \tilde{Q}D \qquad \text{and} \qquad \hat{A} = D^{-1}\tilde{A}D.$$

The meaning of this is that if the first columns of $\hat{Q}$ and $\tilde{Q}$ are essentially the same, then the entire matrices $\hat{Q}$ and $\tilde{Q}$ are essentially the same. After all, the equation $\hat{Q} = \tilde{Q}D$ just means that each column of $\hat{Q}$ is a scalar multiple of the corresponding column of $\tilde{Q}$, and the scalar (some d_k) has modulus 1. The equation $\hat{A} = D^{-1}\tilde{A}D$ means that $\hat{A}$ is essentially the same as $\tilde{A}$. We have $\hat{a}_{ij} = d_i^{-1}d_j\tilde{a}_{ij}$, where $|d_i| = |d_j| = 1$. In summary, a unitary reduction of A to proper upper Hessenberg form is (essentially) uniquely determined by the first column of the transforming matrix.

Proofs of both forms of the implicit-Q theorem are worked out in Exercise 5.7.46.

With the implicit-Q theorem in hand, we can justify the implicit QR step as follows: Suppose we have (by chasing the bulge) transformed the matrix to proper upper Hessenberg form. Suppose further that we have gotten the first column of the transformation matrix right. Then, by the implicit-Q theorem, we must have done a QR iteration. The details are worked out in Exercises 5.7.47 and 5.7.48.

Additional Exercises

Exercise 5.7.25 To find out how well the QR algorithm works without writing your own program, just try out MATLAB's `eig` command, which reduces a matrix to upper Hessenberg form and then applies the implicit QR algorithm. For example, try

```
n = 200;
A = randn(n);
t = cputime;
lambda = eig(A)
et = cputime - t
```

Adjust n to suit the speed of your computer □

Exercise 5.7.26 Show that for each i we have

$$A - \rho I = \hat{Q}_i R_i, \qquad R_i\hat{Q}_i + \rho I = A_i,$$

where A_i, $\hat{Q}_i$, R_i are as in (5.7.4) and (5.7.7). Thus we can view the transformation $A_i = \hat{Q}_i^T A\hat{Q}_i$ as a partial QR iteration based on the partial (or block) QR decomposition $\hat{Q}_i R_i$. We also then have $A_i = R_i A R_i^{-1}$, if R_i is nonsingular. But this is just a variant of (5.7.8). The advantage of (5.7.8) is that it holds regardless of whether or not R_i is invertible. □

Exercise 5.7.27 We used induction to prove that each of the intermediate matrices A_i of (5.7.4) is upper Hessenberg, except for a bulge. Use the formulas $A_i = \hat{Q}_i^T A \hat{Q}_i$ and $A_i R_i = R_i A$ to prove directly that A_i has this form. Specifically:

(a) From the form of $\hat{Q}_i$ deduce that the bottom $n - i - 2$ rows of A_i have the form of an upper Hessenberg matrix.

(b) Prove that the main diagonal entries of the upper triangular matrix $R_{11}^{(i)}$ from (5.7.9) are all nonzero. Thus $R_{11}^{(i)}$ is nonsingular.

(c) Consider the following irregular partition of the equation $A_i R_i = R_i A$.

$$\begin{bmatrix} A_1^{(i)} & A_2^{(i)} \end{bmatrix} \begin{bmatrix} R_{11}^{(i-1)} & * \\ 0 & * \end{bmatrix} = \begin{bmatrix} R_{11}^{(i)} & * \\ 0 & * \end{bmatrix} \begin{bmatrix} A_{11} & A_{12} \\ 0 & A_{22} \end{bmatrix},$$

in which the meaning of A_{ij} is different from before. Now A_{11} is $i \times (i-1)$. Also $A_1^{(i)}$ is $n \times (i-1)$, $R_{11}^{(i-1)}$ is $(i-1) \times (i-1)$, and $R_{11}^{(i)}$ is $i \times i$. Prove that

$$A_1^{(i)} = \begin{bmatrix} R_{11}^{(i)} \\ 0 \end{bmatrix} A_{11} \left[R_{11}^{(i-1)} \right]^{-1}.$$

Prove that the first $i - 1$ columns of A_i have the form of an upper Hessenberg matrix.

(d) Conclude that A_i is upper Hessenberg, except for a bulge at position $(i+2, i)$.

□

Exercise 5.7.28 Prove that for all i the bulge entry $a_{i+2,i}^{(i)}$ in A_i (5.7.4) is nonzero, and the rotator Q_i is nontrivial. □

Exercise 5.7.29 Calculate the product

$$\begin{bmatrix} c & s \\ -s & c \end{bmatrix} \begin{bmatrix} d & g \\ g & e \end{bmatrix} \begin{bmatrix} c & -s \\ s & c \end{bmatrix}.$$

□

Exercise 5.7.30 Consider the symmetric matrix

$$\hat{A} = \begin{bmatrix} d_1 & g_1 & & & & & & & \\ g_1 & d_2 & g_2 & & & & & & \\ & g_2 & d_3 & & & & & & \\ & & & \ddots & & & & & \\ & & & & d_{i-1} & g_{i-1} & b & & \\ & & & & g_{i-1} & d_i & g_i & & \\ & & & & b & g_i & d_{i+1} & & \\ & & & & & & & \ddots & g_{n-1} \\ & & & & & & & g_{n-1} & d_n \end{bmatrix},$$

which would be tridiagonal if it did not have the bulge b in positions $(i+1, i-1)$ and $(i-1, i+1)$. Determine a rotator $\tilde{Q} = \begin{bmatrix} c & -s \\ s & c \end{bmatrix}$ such that $\tilde{Q}^T \begin{bmatrix} g_{i-1} \\ b \end{bmatrix} = \begin{bmatrix} * \\ 0 \end{bmatrix}$. Define $Q_i \in \mathbb{R}^{n \times n}$ by

$$Q_i = \begin{bmatrix} I & 0 & 0 \\ 0 & \tilde{Q} & 0 \\ 0 & 0 & I \end{bmatrix},$$

where $\tilde{Q}$ is embedded in positions i and $i+1$. Calculate $Q_i^T \hat{A} Q_i$, and note that the resulting matrix has a bulge in positions $(i+2, i)$ and $(i, i+2)$; the old bulge is gone. Show that, taking symmetry into account, the transformation from $\hat{A}$ to $Q_i^T \hat{A} Q_i$ requires the calculation of only six new entries. □

Exercise 5.7.31 We continue to use the notation of Exercise 5.7.30.

(a) Suppose $c \neq 0$. Show that $\tilde{Q}$ can be chosen so that $c > 0$. Let $t = s/c$. Show that (if $c > 0$)

$$c^2 = \frac{1}{1+t^2} \qquad s^2 = \frac{t^2}{1+t^2} \qquad cs = \frac{t}{1+t^2}$$

$$c = \frac{1}{\sqrt{1+t^2}} \qquad \text{and} \qquad s = \frac{t}{\sqrt{1+t^2}}.$$

(Keep in mind that the letters s, c, and t stand for sine, cosine, and tangent, respectively. Thus you can use various trigonometric identities such as $c^2 + s^2 = 1$.)

(b) Rewrite the formulas that transform $\hat{A}$ to $Q_i^T \hat{A} Q_i$ entirely in terms of t, omitting all reference to c and s. In particular, notice how simple the formula for t itself is.

(c) Now suppose $s \neq 0$. Show that $\tilde{Q}$ can be chosen so that $s > 0$. Let $k = c/s$. (k stands for *kotangent*.) Show that (if $s > 0$)

$$s^2 = \frac{1}{1+k^2} \qquad c^2 = \frac{k^2}{1+k^2} \qquad cs = \frac{k}{1+k^2}$$

$$s = \frac{1}{\sqrt{1+k^2}} \qquad \text{and} \qquad c = \frac{k}{\sqrt{1+k^2}}.$$

(d) Rewrite the formulas that transform $\hat{A}$ to $Q_i^T \hat{A} Q_i$ entirely in terms of k, omitting all reference to c and s. In particular, notice how simple the formula for k is.

(e) Depending on the relative sizes of b and g_{i-1}, either t or k can be very large. There is a slight danger of overflow. However, k and t can't both be large at the same time. All danger of overflow can be avoided by choosing the appropriate set of formulas for a given step. Show that if $|b| \leq |g_{i-1}|$, then $|t| \leq 1$,

$t^2 \leq 1$, and $1 \leq 1 + t^2 \leq 2$. Show that if $|b| \geq |g_{i-1}|$, then $|k| \leq 1$, $k^2 \leq 1$, and $1 \leq 1 + k^2 \leq 2$.

□

Exercise 5.7.32 Use formulas and ideas from Exercises 5.7.29 and 5.7.31, write an algorithm that implements an implicit QR step on a symmetric, tridiagonal matrix. Try to minimize the number of arithmetic operations. The initial rotator, which creates the first bulge, can be handled in about the same way as the other rotators. □

Numerous ways to organize a symmetric QR step have been proposed. A number of them are discussed in [54] in the context of the equivalent QL algorithm.

Exercise 5.7.33 (Evaluation of the Wilkinson shift)

(a) Show that the eigenvalues of $\begin{bmatrix} d & g \\ g & e \end{bmatrix}$ are the (real) numbers

$$(d+e)/2 \pm \sqrt{[(d-e)/2]^2 + g^2}.$$

The Wilkinson shift is that eigenvalue which is closer to e. Thus we take the positive square root if $e \geq d$ and the negative square root if $e < d$.

(b) Typically g will be small, and the shift will be quite close to e. Thus a stable way to calculate it is by a formula $\rho = e + \delta e$, where δe is a small correction term. Show that $\rho = e + \delta e$, where

$$\delta e = \begin{cases} p + r & \text{if } p \leq 0 \\ p - r & \text{if } p < 0 \end{cases}$$

$$p = \frac{d-e}{2} \qquad \text{and} \qquad r = \sqrt{p^2 + g^2}.$$

Notice that the calculation of δe always involves the addition of two numbers of opposite sign. Thus there will always be some loss of accuracy through cancellation. Fortunately, there is a more accurate way to calculate δe.

(c) Show that

$$p \pm r = \frac{p^2 - r^2}{p \mp r} = \frac{-g^2}{p \mp r}.$$

Thus

$$\delta e = \begin{cases} \frac{g^2}{r-p} & \text{if } p \leq 0 \\ \frac{-g^2}{r+p} & \text{if } p < 0, \end{cases}$$

where p and r are as defined above. In this formulation two positive numbers are always added.

Note: There is some danger of exaggerating the importance of this computation. In fact a shift calculated by a naive application of the quadratic formula will work

almost as well. However, the computation outlined here is no more expensive, so we might as well use it. □

Exercise 5.7.34 Write a Fortran subroutine that calculates the eigenvalues of a real, symmetric, tridiagonal matrix by the implicit QR algorithm. In order to appreciate cubic convergence fully, do all of the computations in double precision.

Since the algorithm requires properly tridiagonal matrices, before each iteration you must check the subdiagonal entries to see whether the problem can be deflated or reduced. In practice any entry g_k that is very close to zero should be regarded as a zero. Use the following criterion: set g_k to zero whenever $|g_k| < u(|d_k| + |d_{k+1}|)$, where u is the unit roundoff error. In IEEE double precision arithmetic, $u \approx 10^{-16}$. This criterion guarantees that the error that is made when g_k is set to zero is comparable in magnitude to the numerous roundoff errors that are made during the computation. In particular, backward stability is not compromised.

Once the matrix has been broken into two or more pieces, the easiest way to keep track of which portions of the matrix still need to be processed is to work from the bottom up. Thus you should check the subdiagonal entries starting from the bottom, and as soon as you find a zero, perform a QR iteration on the bottom submatrix. If you always work on the bottom matrix first, it will be obvious when you are done.

Take full advantage of symmetry. The only storage area you should need is a pair of one dimensional arrays for the main diagonal and subdiagonal, and a few additional single storage locations for temporary variables such as the bulge, tangent, and cotangent.

The usual choice of shift is the Wilkinson shift, since it guarantees convergence. However, since we wish to use this program as a learning tool, build in three shift options: (i) zero shift, (ii) Rayleigh quotient shift, and (iii) Wilkinson shift. Since we wish to observe the convergence of the algorithm, the subroutine should optionally print out the matrix at each iteration and also print out how many iterations are required for each deflation or reduction. Since we wish to observe the convergence of the eigenvalues, print out the *main-diagonal* entries to some sixteen decimal places. Preferably use the exponential format for maximum flexibility. We are interested in only the magnitude of the *subdiagonal* entries, so we do not need to see sixteen digits of the mantissa; two digits are plenty. However, the exponential format is essential.

This subroutine, like all iterative algorithms, must have some limit on the number of iterations allowed. I suggest that you not allow more than $100n$ iterations. (The limit could be set much lower, say $10n$, if only the Wilkinson shift were being used.) If the iteration limit is reached, the subroutine should set an error flag and return.

Be sure to write clear, structured code, document it with a reasonable number of comments, and document clearly all the variables that are passed to and from the subroutine.

Try out your subroutine on the following two matrices using all three shift options.

$$\begin{bmatrix} 16 & 1 & & & & \\ 1 & 8 & 1 & & & \\ & 1 & 4 & 1 & & \\ & & 1 & 2 & 1 & \\ & & & 1 & 1 & \end{bmatrix}.$$

Try the above matrix first, since it converges fairly rapidly, even with zero shifts. The eigenvalues are approximately 16.124, 8.126, 4.244, 2.208, and 0.297. Print out the matrix after each iteration and observe the spectacular cubic convergence of the shifted cases. Now try the 7×7 matrix

$$\begin{bmatrix} 2 & 1 & & & & & \\ 1 & 2 & 1 & & & & \\ & 1 & 2 & 1 & & & \\ & & 1 & 2 & 1 & & \\ & & & 1 & 2 & 1 & \\ & & & & 1 & 2 & 1 \\ & & & & & 1 & 2 \end{bmatrix},$$

whose eigenvalues are $\lambda_k = 4\sin^2(k\pi/16)$, $k = 1, \ldots, 7$. This matrix will cause problems for the Rayleigh quotient shift. Since this matrix requires many iterations for the unshifted case, don't print out the matrix after each iteration. Just keep track of the number of iterations required per eigenvalue.

If you would like to experiment with larger versions of this matrix, the $n \times n$ version has eigenvalues $\lambda_k = 4\sin^2(k\pi/(2n+2))$, $k = 1, \ldots, n$.

You can also test your subroutine's reduction mechanism by concocting some larger matrices with some zeros on the subdiagonal. for example, you can string together three or four copies of the matrices given above. □

Exercise 5.7.35 For those who would prefer not to learn Fortran this week, work Exercise 5.7.34 using MATLAB instead. Your code will be slow, but it will be just as good for instructional purposes. □

Exercise 5.7.36 Show that the computation of the product of two upper Hessenberg matrices requires $O(n^3)$ flops. □

Exercise 5.7.37 This exercise justifies the implicit double QR step. Let $B = (A-\rho I)(A-\tau I)$, let $Q_1, \ldots, Q_{n-1}$ be the reflectors that convert B to upper triangular form: $R = Q_{n-1}^T \cdots Q_2^T Q_1^T B$, let $\hat{Q}_i = Q_1 Q_2 \cdots Q_i$, $A_i = \hat{Q}_i^T A \hat{Q}_i = Q_i^T A_{i-1} Q_i$, and

$$R_i = \hat{Q}_i^T B = \begin{bmatrix} R_{11}^{(i)} & R_{12}^{(i)} \\ 0 & R_{22}^{(i)} \end{bmatrix}, \tag{5.7.38}$$

where $R_{11}^{(i)}$ is an $i \times i$ upper triangular matrix, and $R_{22}^{(i)}$ is a proper 2-Hessenberg matrix. Thus its first column has nonzeros only in the first three positions.

(a) Prove that $A_iR_i = R_iA$.

(b) Show that if A_i is upper Hessenberg, except for a bulge consisting of the entries $a^{(i)}_{i+2,i}$, $a^{(i)}_{i+3,i}$, and $a^{(i)}_{i+3,i+1}$, then

$$\begin{bmatrix} a^{(i)}_{i+1,i} \\ a^{(i)}_{i+2,i} \\ a^{(i)}_{i+3,i} \end{bmatrix} = \beta \begin{bmatrix} r^{(i)}_{i+1,i+1} \\ r^{(i)}_{i+2,i+1} \\ r^{(i)}_{i+3,i+1} \end{bmatrix}$$

for some nonzero scalar β. (*Hint:* Accomplish this by partitioning the equation $A_iR_i = R_iA$ and reasoning as we did in the single-step case.)

(c) Prove by induction on i that A_i is upper Hessenberg, except for a bulge consisting of the entries $a^{(i)}_{i+2,i}$, $a^{(i)}_{i+3,i}$, and $a^{(i)}_{i+3,i+1}$.

This proves that the bulge-chasing process effects a double QR step. □

Exercise 5.7.39 We continue to use the notation from the previous exercise. Deduce the form of A_i directly from the equations.

(a) Use the equation $A_i = \hat{Q}_i^T A \hat{Q}_i$ to deduce that the bottom $n - i - 3$ rows of A_i have the form of an upper Hessenberg matrix.

(b) Prove that the upper triangular matrix $R^{(i)}_{11}$ defined in (5.7.38) is nonsingular if $i \leq n - 2$.

(c) Prove that the first $i - 1$ columns of A_i have the form of an upper Hessenberg matrix. (This is the same as part (c) of Exercise 5.7.27.)

(d) Conclude that A_i is upper Hessenberg, except for a bulge consisting of entries $a^{(i)}_{i+2,i}$, $a^{(i)}_{i+3,i}$, and $a^{(i)}_{i+3,i+1}$.

□

Exercise 5.7.40 Usually the shifts for a double QR step are taken to be the two eigenvalues of the lower right hand 2×2 submatrix

$$\begin{bmatrix} a_{n-1,n-1} & a_{n-1,n} \\ a_{n,n-1} & a_{n,n} \end{bmatrix}.$$

Then either ρ and τ are real or $\tau = \overline{\rho}$.

(a) Show that if we choose the shifts this way, we have

$$\begin{aligned} \rho + \tau &= a_{n-1,n-1} + a_{n,n} && (= \text{trace}) \\ \rho\tau &= a_{n-1,n-1}a_{n,n} - a_{n,n-1}a_{n-1,n} && (= \text{determinant}) \end{aligned}$$

(b) Show that the first column of $B = (A - \tau I)(A - \rho I)$ is proportional to

$$\begin{aligned} v_1 &= \frac{(a_{11} - a_{n-1,n-1})(a_{11} - a_{n,n}) - a_{n,n-1}a_{n-1,n}}{a_{21}} + a_{12} \\ v_2 &= a_{11} + a_{22} - a_{n-1,n-1} - a_{n,n} \\ v_3 &= a_{32}. \end{aligned}$$

This is a good formula to use for computing the initial reflector Q_1. □

Exercise 5.7.41 Given a nonzero $y \in \mathbb{R}^3$, let $Q \in \mathbb{R}^{3\times 3}$ be the reflector such that

$$Q\begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} -\tau \\ 0 \\ 0 \end{bmatrix} \quad \text{and} \quad \tau = \begin{cases} \|y\|_2 & \text{if } y_1 \geq 0 \\ -\|y\|_2 & \text{if } y_1 < 0. \end{cases}$$

(a) (Review) Show that $Q = I - \gamma u u^T$, where

$$u = \begin{bmatrix} 1 \\ y_2/(y_1+\tau) \\ y_3/(y_1+\tau) \end{bmatrix}^T$$

and $\gamma = (y_1 + \tau)/\tau$. Thus $Q = I - vu^T$, where $v = \gamma u$.

(b) Let $B \in \mathbb{R}^{3\times k}$ be a submatrix to be transformed by Q. Noting that $u_1 = 1$, show that the operation $B \to QB = B - v(u^T B)$ requires only $5k$ multiplications and $5k$ additions. (Since $Q = Q^T = I - uv^T$, operations of the type $C \to CQ$ can be performed in exactly the same way.)

(c) Show that if the operations are carried out as indicated in part (b), a double implicit QR step requires only about $5n^2$ multiplications and $5n^2$ additions, that is, $10n^2$ flops.

□

Exercise 5.7.42 Write a pseudocode algorithm to perform an implicit double QR step using reflectors. □

Exercise 5.7.43 Write a MATLAB m-file that executes an implicit double QR step. You can get test examples in various ways in MATLAB. For example, you can build a random matrix and have MATLAB's `hess` command transform it to upper Hessenberg form. Build a sample matrix and save a copy. Iterate with your m-file until you have isolated a pair of eigenvalues. What evidence of quadratic convergence do you see? Since MATLAB has an `eig` command, you can easily check whether your eigenvalues are correct. □

Exercise 5.7.44 Write a Fortran subroutine that calculates the eigenvalues and eigenvectors of an upper Hessenberg matrix by the implicit double-step QR algorithm. In order to appreciate quadratic convergence fully, do all of the computations in double precision. Since the algorithm requires properly upper Hessenberg matrices, before each iteration you must check the subdiagonal entries to see whether the problem can be deflated or reduced. In practice any entry $a_{k+1,k}$ that is very close to zero should be regarded as a zero. Use the following criterion: set $a_{k+1,k}$ to zero whenever $|a_{k+1,k}| < u(|a_{k,k}| + |a_{k+1,k+1}|)$, where u is the unit roundoff error. In IEEE double precision arithmetic, $u \approx 10^{-16}$. This criterion guarantees that the error that is made when $a_{k+1,k}$ is set to zero is comparable in magnitude to the numerous

roundoff errors that are made during the computation. In particular, backward stability is not compromised.

Once the matrix has been broken into two or more pieces, the easiest way to keep track of which portions of the matrix still need to be processed is to work from the bottom up. Thus you should check the subdiagonal entries starting from the bottom, and as soon as you find a zero, perform a QR iteration on the bottom submatrix. If you always work on the bottom matrix first, it will be obvious when you are done. An isolated 1×1 block is a real eigenvalue. An isolated 2×2 block contains a pair of complex or real eigenvalues that can be found (carefully) by the quadratic formula. (See Exercise 5.6.27). Once you get to the top of the matrix, you are done.

Since we wish to observe the quadratic convergence of the algorithm, the subroutine should optionally print out the subdiagonal of the matrix after each iteration. It should also print out how many iterations are required for each deflation or reduction.

The algorithm should have a limit on the number of steps allowed, and an exceptional shift capability should be built in to deal with stubborn cases.

Write clear, structured code, document it with a reasonable number of comments, and document clearly all of the variables that are passed to and from the subroutine.

Try out your subroutine on the matrix of Example 5.6.18, whose eigenvalues are $\cos(2\pi j/n) + i\sin(2\pi j/n)$, $j = 1, \ldots, n$. Try several values of n. Also try the symmetric test matrices of Exercise 5.7.34. Finally, you might like to try the matrix given in [83], page 370, which has some ill-conditioned eigenvalues. □

Exercise 5.7.45 Let $x \in \mathbb{C}^{n\times n}$ be a nonzero vector, and let $e_1 \in \mathbb{C}^{n\times n}$ be the first standard unit vector. In many contexts we need to generate nonsingular transforming matrices G that "introduce zeros" into the vector x, in the sense that $G^{-1}x = \beta e_1$, where β is a nonzero constant. Transformations of this type are the heart of LU and QR decompositions, for example, and have found extensive use in this section. Show that $G^{-1}x = \beta e_1$ for some β if and only if the first column of G is proportional to x. □

Exercise 5.7.46 This exercise leads to proofs of both versions of the implicit-Q theorem. Given a matrix $A \in \mathbb{C}^{n\times n}$ and a vector $x \in \mathbb{C}^n$, we define the *Krylov matrix* $K(A, x) = K_n(A, x) \in \mathbb{C}^{n\times n}$ to be the matrix whose columns are x, Ax, A^2x, ..., $A^{n-1}x$. Krylov matrices and the associated *Krylov subspaces* will play an important role in Chapters 6 and 7.

(a) Let $B \in \mathbb{C}^{n\times n}$ be an upper Hessenberg matrix, and let e_1 be the first standard unit vector (first column of the identity matrix). Prove that $K(B, e_1)$ is upper triangular. Prove that if B is properly upper Hessenberg, then $K(B, e_1)$ is nonsingular. Prove that if the subdiagonal entries $b_{j+1,j}$, $j = 1, \ldots, n-1$ are all positive, then the main-diagonal entries of $K(B, e_1)$ are all positive.

(b) Show that if $B = Q^{-1}AQ$, then $K(A, Qe_1) = QK(B, e_1)$. Under what conditions on Q and B can this result be interpreted as a QR decomposition?

(c) Let $\hat{Q}$ and $\tilde{Q}$ be unitary matrices that have the same first column ($\hat{Q}e_1 = \tilde{Q}e_1$), and let $\hat{A} = \hat{Q}^*A\hat{Q}$ and $\tilde{A} = \tilde{Q}^*A\tilde{Q}$. Suppose $\hat{A}$ and $\tilde{A}$ are both properly

upper Hessenberg matrices with $\hat{a}_{j+1,j} > 0$, $\tilde{a}_{j+1,j} > 0$, $j = 1, \ldots, n-1$. Using the result from part (b) and the uniqueness of the QR decomposition, show that $\hat{Q} = \tilde{Q}$ and $\hat{A} = \tilde{A}$. This is Theorem 5.7.23, the strict from of the implicit-Q theorem.

(d) Now relax the conditions. Suppose $\hat{Q}e_1 = \tilde{Q}e_1 d_1$, $\hat{A}$ is properly upper Hessenberg, and $\tilde{A}$ is upper Hessenberg. Using the result from part (b) and the essential uniqueness of the QR decompositions (cf. Exercise 3.2.60), show that $\tilde{A}$ must also be properly upper Hessenberg, and $\hat{Q} = \tilde{Q}D$, where D is a diagonal matrix with diagonal entries d_k satisfying $|d_k| = 1$. This is Theorem 5.7.24, the relaxed version of the implicit-Q theorem.

□

Exercise 5.7.47 This exercise gives a more concise introduction to and justification of the implicit single QR step. The basic idea is that if you do a transformation to upper Hessenberg form that has the right first column, then you've done a QR step. Define Q_1 as in (5.7.5) and (5.7.6). Then $Q_1^T A Q_1$ is not upper Hessenberg, as it has a bulge in the $(3,1)$ position. Let $Q_2, \ldots, Q_{n-1}$ be orthogonal transformations that return $Q_1^T A Q_1$ to upper Hessenberg form by chasing the bulge. Let $\tilde{Q} = Q_1 Q_2 \cdots Q_{n-1}$ and $\tilde{A} = \tilde{Q}^T A \tilde{Q}$, an upper Hessenberg matrix. With the construction of $\tilde{A}$, the implicit QR iteration is complete. The explicit QR step is given by $\hat{A} = Q^T A Q$ (5.7.2), where Q is given by the first equation of (5.7.1).

(a) Show that first column of $\tilde{Q}$ is the same as the first column of Q_1.

(b) Show that the first column of $\tilde{Q}$ and the first column of Q are both proportional to the first column of $A - \rho I$.

(c) Suppose $\hat{A}$ is a proper upper Hessenberg matrix. Using Theorem 5.7.24, show that $Q = \tilde{Q}D$ and $\hat{A} = D^{-1}\tilde{A}D$, where D is a diagonal matrix whose main-diagonal entries satisfy $|d_i| = 1$. Thus the explicit and implicit QR steps produce essentially the same result. (With a bit more effort, the assumption that $\hat{A}$ is properly Hessenberg can be removed.)

□

Exercise 5.7.48 This exercise gives a concise justification of the implicit double QR step. Let ρ and τ be shifts that are either real or satisfy $\overline{\tau} = \rho$. Let $B = (A - \tau I)(A - \rho I)$. Then B is real, and its first column has only three nonzero entries, which are given by (5.7.22). Let Q_1 be an orthogonal matrix (e.g., a reflector) acting on rows 1–3, whose first column is a multiple of the first column of B, and let $A_1 = Q_1^T A Q_1$. Let Q_2, $\ldots$, Q_{n-1} be orthogonal transformations that return $Q_1^T A Q_1$ to upper Hessenberg form by chasing the bulge. Let $\tilde{Q} = Q_1 Q_2 \cdots Q_{n-1}$ and $\tilde{A} = \tilde{Q}^T A \tilde{Q}$, an upper Hessenberg matrix. With the construction of $\tilde{A}$, the implicit double QR iteration is complete. The explicit double QR step is given by $\hat{A} = Q^T A Q$, where Q is the orthogonal factor in the decomposition $(A - \tau I)(A - \rho I) = QR$.

(a) Show that first column of $\tilde{Q}$ is the same as the first column of Q_1.

(b) Show that the first column of $\tilde{Q}$ and the first column of Q are both proportional to the first column of B.

(c) Suppose $\hat{A}$ is a proper upper Hessenberg matrix. Using Theorem 5.7.24, show that $Q = \tilde{Q}D$ and $\hat{A} = D^{-1}\tilde{A}D$, where D is a diagonal matrix whose main-diagonal entries satisfy $|d_i| = 1$. Thus the explicit and implicit double QR steps produce essentially the same result. (Again the assumption that $\hat{A}$ is properly Hessenberg can be removed with a bit more work.)

□

5.8 USE OF THE QR ALGORITHM TO CALCULATE EIGENVECTORS

In Section 5.3 we observed that inverse iteration can be used to find eigenvectors associated with known eigenvalues. This is a powerful and important technique. In this section we will see how the QR algorithm can be used to calculate eigenvalues and eigenvectors simultaneously. In Section 6.2 it will be shown that this use of the QR algorithm can be viewed as a form of inverse iteration. We will begin by discussing the symmetric case, since it is relatively uncomplicated.

Symmetric Matrices

Let $A \in \mathbb{R}^{n\times n}$ be symmetric and tridiagonal. The QR algorithm can be used to find all of the eigenvalues and eigenvectors of A at once. After some finite number of shifted QR steps, A will have been reduced essentially to diagonal form

$$D = Q^T A Q, \tag{5.8.1}$$

where the main-diagonal entries of D are the eigenvalues of A. If a total of m QR steps are taken, then $Q = Q_1 \cdots Q_m$, where Q_i is the transforming matrix for the ith step. Each Q_i is a product of $n-1$ rotators. Thus Q is the product of a large number of rotators. The importance of Q is that its columns are a complete orthonormal set of eigenvectors of A, by Spectral Theorem 5.4.12. It is easy to accumulate Q in the course of performing the QR algorithm, thereby obtaining the eigenvectors along with the eigenvalues. An additional array is needed for the accumulation of Q. Calling this array Q also, we set $Q = I$ initially. Then for each rotator Q_{ij} that is applied to A (that is, $A \leftarrow Q_{ij}^T A Q_{ij}$), we multiply Q by Q_{ij} on the right ($Q \leftarrow QQ_{ij}$). The end result is clearly the transforming matrix Q of (5.8.1).

How much does this transformation procedure cost? Each transformation $Q \leftarrow QQ_{ij}$ alters two columns of Q. Thus $2n$ numbers are updated. The exact flop count depends upon the details of the implementation, but in any event it is $O(n)$. Since each QR step uses $n-1$ rotators, the cost of updating Q for each complete step is $O(n^2)$ flops. Recalling that the basic cost of a symmetric QR step is $O(n)$ flops,

we conclude that the accumulation of Q increases the cost by an order of magnitude. Making the (reasonable) assumption that the total number of QR iterations is $O(n)$, the total cost of accumulating Q is $O(n^3)$ flops, whereas the total cost of the QR iterations without accumulating Q is $O(n^2)$.

An obvious way to decrease the cost of accumulating Q is to decrease the number of QR iterations. A modest decrease can be achieved by the *ultimate shift* strategy: First use QR without accumulating Q to calculate the eigenvalues only. This is cheap, only $O(n)$ flops per iteration. Then (having saved a copy of A) perform the QR algorithm over again, accumulating Q, this time using the computed eigenvalues as shifts. With these excellent shifts, the total number of QR steps needed to reduce the matrix to diagonal form is reduced, so time is saved by accumulating Q on the second pass. In principle each QR step should deliver an eigenvalue (Corollary 5.6.23). Because of roundoff errors, two steps are needed for most eigenvalues, but the number of iterations is reduced nevertheless. The performance of the algorithm depends upon the order in which the eigenvalues are applied as shifts. A good order to use is the order in which the eigenvalues emerged on the first pass.

Exercise 5.8.2 Assuming k QR iterations are needed for each eigenvalue and taking deflation into account, show that Q is the product of about $\frac{1}{2}kn^2$ rotators. (If we use the ultimate shift strategy, then k is around 2. Otherwise it is closer to 3.) □

Exercise 5.8.3 Outline the steps that would be taken to calculate a complete set of eigenvalues and eigenvectors of a symmetric (non-tridiagonal) matrix using the QR algorithm. Estimate the flop count for each step and the total flop count. □

Exercise 5.8.4 Write a Fortran program that calculates the eigenvectors of a symmetric tridiagonal matrix by the QR algorithm. □

Unsymmetric Matrices

Let $A \in \mathbb{C}^{n\times n}$ be an upper Hessenberg matrix. If the QR algorithm is used to calculate the eigenvalues of A, then after some finite number of steps we have essentially

$$T = Q^*AQ, \tag{5.8.5}$$

where T is upper triangular. This is the Schur form (Theorem 5.4.11). As in the symmetric case, we can accumulate the transforming matrix Q. Again we might consider using the ultimate shift strategy, in which we calculate the eigenvalues first without accumulating Q, then run the QR algorithm again, using the computed eigenvalues as shifts, and accumulating Q. In this case the basic cost of a QR step is $O(n^2)$, not $O(n)$, so there is a significant overhead associated with performing the task in two passes. However, the savings realized by accumulating Q on the second pass are normally more than enough to offset the overhead.

There is an important difference in the way the QR algorithm is handled, depending upon whether just eigenvalues are computed or eigenvectors as well. If a zero appears on the subdiagonal of some iterate, then the matrix has the block-diagonal

form

$$\begin{bmatrix} A_{11} & A_{12} \\ 0 & A_{22} \end{bmatrix}.$$

If only eigenvalues are needed, then A_{11} and A_{22} can be treated separately, and A_{12} can be ignored. However, if eigenvectors are wanted, then A_{12} cannot be ignored. We must continue to update it because it eventually forms part of the matrix T, which is needed for the computation of the eigenvectors. Thus if rows i and j of A_{11} are altered by some rotator, then the same rotator must be applied to rows i and j of A_{12}. Similarly, if columns i and j of A_{22} are altered by some rotator, then the corresponding columns of A_{12} must also be updated.

Once we have obtained the form (5.8.5), we are still not finished. Only the first column of Q is an eigenvector of A. To obtain the other eigenvectors, we must do a bit more work. It suffices to find the eigenvectors of T, since for each eigenvector v of T, Qv is an eigenvector of A. Let us therefore examine the problem of calculating the eigenvectors of an upper-triangular matrix. The eigenvalues of T are t_{11}, t_{22}, $\ldots$, t_{nn}, which we will assume to be distinct. To find an eigenvector associated with the eigenvalue t_{kk}, we must solve the homogeneous equation $(T - t_{kk}I)v = 0$. It is convenient to make the partition

$$T - t_{kk}I = \begin{bmatrix} S_{11} & S_{12} \\ 0 & S_{22} \end{bmatrix}, \qquad v = \begin{bmatrix} v_1 \\ v_2 \end{bmatrix},$$

where $S_{11} \in \mathbb{C}^{k\times k}$ and $v_1 \in \mathbb{C}^k$. Then the equation $(T - t_{kk}I)v = 0$ becomes

$$\begin{aligned} S_{11}v_1 + S_{12}v_2 &= 0 \\ S_{22}v_2 &= 0. \end{aligned}$$

S_{11} and S_{22} are upper triangular. S_{22} is nonsingular, because its main-diagonal entries are $t_{jj} - t_{kk}$, $j = k+1, \ldots, n$, all of which are nonzero. Therefore v_2 must equal zero, and the equations reduce to $S_{11}v_1 = 0$. S_{11} is singular, because its (k,k) entry is zero. Making another partition

$$S_{11} = \begin{bmatrix} \hat{S} & r \\ 0^T & 0 \end{bmatrix}, \qquad v_1 = \begin{bmatrix} \hat{v} \\ w \end{bmatrix},$$

where $\hat{S} \in \mathbb{C}^{(k-1)\times(k-1)}$, $r \in \mathbb{C}^{k-1}$, and $\hat{v} \in \mathbb{C}^{k-1}$, the equation $S_{11}v_1 = 0$ becomes

$$\hat{S}\hat{v} + rw = 0.$$

The matrix $\hat{S}$ is upper triangular and nonsingular. Taking w to be any nonzero number, we can solve $\hat{S}\hat{v} = -rw$ for $\hat{v}$ by back substitution to obtain an eigenvector. For example, if we take $w = 1$, we get the eigenvector

$$v = \begin{bmatrix} -\hat{S}^{-1}r \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

Exercise 5.8.6 Calculate three linearly independent eigenvectors of the matrix

$$T = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 2 & 1 \\ 0 & 0 & 3 \end{bmatrix}.$$

□

Exercise 5.8.7 How many flops are needed to calculate n linearly independent eigenvectors of the upper-triangular matrix $T \in \mathbb{C}^{n \times n}$ with distinct eigenvalues? □

The case in which T has repeated eigenvalues is more complicated, since T may fail to have n linearly independent eigenvectors; that is, T may be defective. The case of repeated eigenvalues is worded out, in part, in Exercise 5.8.8. Even when the eigenvalues are distinct, the algorithm we have just outlined can sometimes give inaccurate results, because the eigenvectors can be ill conditioned. Section 6.5 discusses conditioning of eigenvalues and eigenvectors.

Exercise 5.8.8 Let T be an upper-triangular matrix with a double eigenvalue $\lambda = t_{jj} = t_{kk}$. Give necessary and sufficient conditions under which T has two linearly independent eigenvectors associated with λ, and outline a procedure for calculating the eigenvectors. □

If A is a real matrix, we prefer to work within the real number system as much as possible. Therefore we use the double QR algorithm to reduce A to the form

$$T = Q^T A Q,$$

where $Q \in \mathbb{R}^{n \times n}$ is orthogonal, and $T \in \mathbb{R}^{n \times n}$ is quasi-triangular. This is the Wintner-Murnaghan form (Theorem 5.4.22).

Exercise 5.8.9 Sketch a procedure for calculating the eigenvectors of a quasi-triangular matrix $T \in \mathbb{R}^{n \times n}$ that has distinct eigenvalues. □

Suppose $A \in \mathbb{R}^{n \times n}$ has complex eigenvalues λ and $\overline{\lambda}$. Since the associated eigenvectors are complex, we expect that we will have to deal with complex vectors at some point. In the name of efficiency we would like to postpone the use of complex arithmetic for as long as possible. It turns out that we can avoid complex arithmetic right up to the very end. Suppose $A = QTQ^T$, where T is the quasi-triangular Wintner-Murnaghan form. Using the procedure you sketched in Exercise 5.8.9, we can find an eigenvector v of T associated with λ: $Tv = \lambda v$. Then $w = Qv$ is an eigenvector of A. Both v and w can be broken into real and imaginary parts: $v = v_1 + iv_2$ and $w = w_1 + iw_2$, where v_1, v_2, w_1, and $w_2 \in \mathbb{R}^n$. Since Q is real, it follows easily that $w_1 = Qv_1$ and $w_2 = Qv_2$. Thus the computation of w does not require complex arithmetic; w_1 and w_2 can be calculated individually, entirely in real arithmetic. Notice also that $\overline{w} = w_1 - iw_2$ is the eigenvector of A associated with $\overline{\lambda}$. Thus the two real vectors w_1 and w_2 yield two complex eigenvectors.

Exercise 5.8.10 Rework Exercise 5.8.9 so that all calculations are done in real arithmetic. □

5.9 THE SVD REVISITED

Throughout this section, A will denote a nonzero, real $n \times m$ matrix. In Chapter 4 we introduced the singular value decomposition (SVD) and proved that A has an SVD. The proof given there (Exercise 4.1.17) does not use the concepts of eigenvalue and eigenvector. Later on (Exercise 5.2.17) we noticed that singular values and vectors are closely related to eigenvalues and eigenvectors. In this section we will present a second proof of the SVD theorem that makes use of the eigenvector connection. Then we will show how to compute the SVD.

A Second Proof of the SVD Theorem

The following development does not depend on any of the results from Chapter 4. Recall that $A \in \mathbb{R}^{n\times m}$ has two important spaces associated with it—the *null space* and the *range*, given by

$$\mathcal{N}(A) = \{x \in \mathbb{R}^m \mid Ax = 0\},$$

$$\mathcal{R}(A) = \{Ax \mid x \in \mathbb{R}^m\}.$$

The null space is a subspace of $\mathbb{R}^m$, and the range is a subspace of $\mathbb{R}^n$. Recall that the range is also called the column space of A (Exercise 3.5.13), and its dimension is called the *rank* of A. Finally, recall that $m = \dim(\mathcal{N}(A)) + \dim(\mathcal{R}(A))$ This is Corollary 4.1.9, which can also be proved by elementary means.

The matrices $A^TA \in \mathbb{R}^{m\times m}$ and $AA^T \in \mathbb{R}^{n\times n}$ will play an important role in what follows. Let us therefore explore the properties of these matrices and their relationships to A and A^T.

Exercise 5.9.1 **(Review)** Prove that A^TA and AA^T are (a) symmetric and (b) positive semidefinite. □

Theorem 5.9.2 $\mathcal{N}(A^TA) = \mathcal{N}(A)$.

Proof. It is obvious that $\mathcal{N}(A) \subseteq \mathcal{N}(A^TA)$. To prove that $\mathcal{N}(A^TA) \subseteq \mathcal{N}(A)$, suppose $x \in \mathcal{N}(A^TA)$. Then $A^TAx = 0$. An easy computation (Lemma 3.5.9) shows that $\langle A^TAx, x\rangle = \langle Ax, Ax\rangle$. Thus $0 = \langle A^TAx, x\rangle = \langle Ax, Ax\rangle = \|Ax\|_2^2$. Therefore $Ax = 0$; that is, $x \in \mathcal{N}(A)$. □

Corollary 5.9.3 $\mathcal{N}(AA^T) = \mathcal{N}(A^T)$.

Corollary 5.9.4 $\text{rank}(A^TA) = \text{rank}(A) = \text{rank}(A^T) = \text{rank}(AA^T)$.

Proof. By Corollary 4.1.9, $\text{rank}(A) = m - \dim(\mathcal{N}(A))$ and, similarly, $\text{rank}(A^TA) = m - \dim(\mathcal{N}(A^TA))$. Thus $\text{rank}(A^TA) = \text{rank}(A)$. The second equation is a basic result of linear algebra. The third equation is the same as the first, except that the roles of A and A^T have been reversed. □

Proposition 5.9.5 *If v is an eigenvector of A^TA associated with a nonzero eigenvalue λ, then Av is an eigenvector of AA^T associated with the same eigenvalue.*

Proof. Since $\lambda \neq 0$, we have $Av \neq 0$. Furthermore $AA^T(Av) = A(A^TAv) = A(\lambda v) = \lambda(Av)$. □

Corollary 5.9.6 *A^TA and AA^T have the same nonzero eigenvalues, counting multiplicity.*

Proposition 5.9.7 *Let v_1 and v_2 be eigenvectors of A^TA. If v_1 and v_2 are orthogonal, then Av_1 and Av_2 are also orthogonal.*

Exercise 5.9.8 Prove Proposition 5.9.7. □

Exercise 5.9.9 Let $B \in \mathbb{C}^{m\times m}$ be a semisimple matrix with linearly independent eigenvectors $v_1, \ldots, v_m \in \mathbb{C}^m$, associated with eigenvalues $\lambda_1, \ldots, \lambda_m \in \mathbb{C}$. Suppose $\lambda_1, \ldots, \lambda_r$ are nonzero and $\lambda_{r+1}, \ldots, \lambda_m$ are zero. Show that $\mathcal{N}(B) = \text{span}\{v_{r+1}, \ldots, v_m\}$ and $\mathcal{R}(B) = \text{span}\{v_1, \ldots, v_r\}$. Therefore $\text{rank}(B) = r$, the number of nonzero eigenvalues. □

The matrices A^TA and AA^T are both symmetric and hence semisimple. Thus Corollary 5.9.6 and Exercise 5.9.9 yield a second proof that they have the same rank, which equals the number of nonzero eigenvalues. Since A^TA and AA^T are generally not of the same size, they cannot have exactly the same eigenvalues. The difference is made up by a zero eigenvalue of the appropriate multiplicity. If $\text{rank}(A^TA) = \text{rank}(AA^T) = r$, and $r < m$, then A^TA has a zero eigenvalue of multiplicity $m - r$. If $r < n$, then AA^T has a zero eigenvalue of multiplicity $n - r$.

Exercise 5.9.10 (a) Give an example of a matrix $A \in \mathbb{R}^{1\times 2}$ such that A^TA has a zero eigenvalue and AA^T does not. (b) Where does the proof of Proposition 5.9.5 break down in the case $\lambda = 0$? □

By now we are used to the idea that if a matrix is symmetric, then we can find a complete set of orthonormal eigenvectors. Up to now we have not shown that the orthogonality is essentially unavoidable.

Proposition 5.9.11 *Let $B \in \mathbb{R}^{n\times n}$ be a symmetric matrix with eigenvectors u_i and u_j associated with eigenvalues λ_i and λ_j, with $\lambda_i \neq \lambda_j$. Then u_i and u_j are orthogonal.*

Proof. Applying Lemma 3.5.9 and using $A = A^T$, we have $\langle Au_i, u_j\rangle = \langle u_i, Au_j\rangle$. Therefore

$$\lambda_i\langle u_i, u_j\rangle = \langle \lambda_i u_i, u_j\rangle = \langle Au_i, u_j\rangle = \langle u_i, Au_j\rangle = \langle u_i, \lambda_j u_j\rangle = \lambda_j\langle u_i, u_j\rangle.$$

Taking the difference of the left and right-hand sides of this string of equations, we have

$$(\lambda_i - \lambda_j)\langle u_i, u_j\rangle = 0.$$

Since $\lambda_i - \lambda_j \neq 0$, it must be that $\langle u_i, u_j \rangle = 0$. □

The following theorem is the same as Theorem 4.1.3, except that one sentence has been added.

Theorem 5.9.12 *(Geometric SVD Theorem) Let $A \in \mathbb{R}^{n \times m}$ be a nonzero matrix with rank r. Then $\mathbb{R}^m$ has an orthonormal basis $v_1, \ldots, v_m$, $\mathbb{R}^n$ has an orthonormal basis $u_1, \ldots, u_n$, and there exist $\sigma_1 \geq \sigma_2 \geq \ldots \geq \sigma_r > 0$ such that*

$$Av_i = \begin{cases} \sigma_i u_i & i = 1, \ldots, r \\ 0 & i = r+1, \ldots, m \end{cases} \qquad A^T u_i = \begin{cases} \sigma_i v_i & i = 1, \ldots, r \\ 0 & i = r+1, \ldots, n \end{cases} \tag{5.9.13}$$

Equations (5.9.13) imply that $v_1, \ldots, v_m$ are eigenvectors of $A^T A$, $u_1, \ldots, u_n$ are eigenvectors of AA^T, and $\sigma_1^2, \ldots, \sigma_r^2$ are the nonzero eigenvalues of $A^T A$ and AA^T.

Proof. The final sentence is essentially a rerun of Exercise 5.2.17. You can easily verify that its assertions are true. This determines how $v_1, \ldots, v_m$ must be chosen. Let $v_1, \ldots, v_m$ be an orthonormal basis of $\mathbb{R}^m$ consisting of eigenvectors of $A^T A$. Corollary 5.4.21 guarantees the existence of this basis. Let $\lambda_1, \ldots, \lambda_m$ be the associated eigenvalues. Since $A^T A$ is positive semidefinite, all of its eigenvalues are real and nonnegative. Assume $v_1, \ldots, v_m$ are ordered so that $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_m$. Since $r = \operatorname{rank}(A^T A)$, it must be that $\lambda_r > 0$ and $\lambda_{r+1} = \cdots = \lambda_m = 0$. For $i = 1, \ldots, r$, define $\sigma_i \in \mathbb{R}$ and $u_i \in \mathbb{R}^n$ by

$$\sigma_i = \| Av_i \|_2 \qquad \text{and} \qquad u_i = \frac{1}{\sigma_i} Av_i.$$

These definitions imply that $Av_i = \sigma_i u_i$ and $\| u_i \|_2 = 1$, $i = 1, \ldots, r$. Proposition 5.9.7 implies that $u_1, \ldots, u_r$ are orthogonal, hence orthonormal.

It is easy to show that $\sigma_i^2 = \lambda_i$ for $i = 1, \ldots, r$. Indeed $\sigma_i^2 = \| Av_i \|_2^2 = \langle Av_i, Av_i \rangle = \langle A^T Av_i, v_i \rangle = \langle \lambda_i v_i, v_i \rangle = \lambda_i$. It now follows easily that $A^T u_i = \sigma_i v_i$, for $A^T u_i = (1/\sigma_i) A^T Av_i = (\lambda_i / \sigma_i) v_i = \sigma_i v_i$.

The proof is now complete, except that we have not defined $u_{r+1}, \ldots, u_n$, assuming $r < n$. By Proposition 5.9.5, the vectors $u_1, \ldots, u_r$ are eigenvectors of AA^T associated with nonzero eigenvalues. Since $AA^T \in \mathbb{R}^{n \times n}$ and $\operatorname{rank}(AA^T) = r$, AA^T must have a null space of dimension $n - r$ (Corollary 4.1.9). Let $u_{r+1}, \ldots, u_n$ be any orthonormal basis of $\mathcal{N}(AA^T)$. Noting that $u_{r+1}, \ldots, u_n$ are all eigenvectors of AA^T associated with the eigenvalue zero, we see that each of the vectors $u_{r+1}, \ldots, u_n$ is orthogonal to each of the vectors $u_1, \ldots, u_r$ (Proposition 5.9.11). Thus $u_1, \ldots, u_n$ is an orthonormal basis of $\mathbb{R}^n$ consisting of eigenvectors of AA^T. Since $\mathcal{N}(AA^T) = \mathcal{N}(A^T)$, we have $A^T u_i = 0$ for $i = r+1, \ldots, n$. This completes the proof. □

Recall from Chapter 4 that the numbers $\sigma_1, \ldots, \sigma_r$ are called the (nonzero) *singular values* of A. These numbers are uniquely determined, since they are the positive square roots of the nonzero eigenvalues of $A^T A$. The vectors $v_1, \ldots, v_m$ are *right singular vectors* of A, and $u_1, \ldots, u_n$ are *left singular vectors* of A. Singular

vectors are not uniquely determined; they are no more uniquely determined than any eigenvectors of length 1. Any singular vector can be replaced by its opposite, and if A^TA or AA^T happens to have some repeated eigenvalues, an even more serious lack of uniqueness results.

Computing the SVD

One way to compute the SVD of A is simply to calculate the eigenvalues and eigenvectors of A^TA and AA^T. This approach is illustrated in the following example and exercises. After that we will discuss other, more accurate, approaches, in which the SVD is computed without forming A^TA or AA^T explicitly.

Example 5.9.14 Find the singular values and right and left singular vectors of the matrix

$$A = \begin{bmatrix} 1 & 2 & 0 \\ 2 & 0 & 2 \end{bmatrix}.$$

Since A^TA is 3×3 and AA^T is 2×2, it seems reasonable to work with the latter. We easily compute

$$AA^T = \begin{bmatrix} 5 & 2 \\ 2 & 8 \end{bmatrix},$$

so the characteristic polynomial is $(\lambda-5)(\lambda-8)-4 = \lambda^2-13\lambda+36 = (\lambda-9)(\lambda-4)$, and the eigenvalues of AA^T are $\lambda_1 = 9$ and $\lambda_2 = 4$. The singular values of A are therefore

$$\sigma_1 = 3 \qquad \text{and} \qquad \sigma_2 = 2.$$

The left singular vectors of A are eigenvectors of AA^T. Solving $(\lambda_1 I - AA^T)u = 0$, we find that multiples of $[1,\ 2]^T$ are eigenvectors of AA^T associated with λ_1. Then solving $(\lambda_2 I - AA^T)u = 0$, we find that the eigenvectors of AA^T corresponding to λ_2 are multiples of $[2,\ -1]^T$. Since we want representatives with unit Euclidean norm, we take

$$u_1 = \frac{1}{\sqrt{5}}\begin{bmatrix} 1 \\ 2 \end{bmatrix} \qquad \text{and} \qquad u_2 = \frac{1}{\sqrt{5}}\begin{bmatrix} 2 \\ -1 \end{bmatrix}.$$

(What other choices could have been made?) These are the left singular vectors of A. Notice that they are orthogonal, as they must be.

We can find the right singular vectors v_1, v_2, and v_3 by calculating the eigenvectors of A^TA. However v_1 and v_2 are more easily found by the formula $v_i = \sigma_i^{-1}A^Tu_i, \quad i = 1, 2$. Thus

$$v_1 = \frac{1}{3\sqrt{5}}\begin{bmatrix} 5 \\ 2 \\ 4 \end{bmatrix} \qquad \text{and} \qquad v_2 = \frac{1}{\sqrt{5}}\begin{bmatrix} 0 \\ 2 \\ -1 \end{bmatrix}.$$

Notice that these vectors are orthonormal. The third vector must satisfy $Av_3 = 0$. Solving the equation $Av = 0$ and normalizing the solution, we get

$$v_3 = \frac{1}{3}\begin{bmatrix} -2 \\ 1 \\ 2 \end{bmatrix}.$$

In this case we could have found v_3 without reference to A by applying the Gram-Schmidt process to find a vector orthogonal to both v_1 and v_2. Normalizing the vector, we would get $\pm v_3$.

Now that we have the singular values and singular vectors of A, we can easily construct the SVD $A = U\Sigma V^T$ with $U \in \mathbb{R}^{2\times 2}$ and $V \in \mathbb{R}^{3\times 3}$ orthogonal and $\Sigma \in \mathbb{R}^{2\times 2}$ diagonal. We have

$$U = \begin{bmatrix} u_1 & u_2 \end{bmatrix} = \frac{1}{\sqrt{5}}\begin{bmatrix} 1 & 2 \\ 2 & -1 \end{bmatrix},$$

$$\Sigma = \begin{bmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \end{bmatrix} = \begin{bmatrix} 3 & 0 & 0 \\ 0 & 2 & 0 \end{bmatrix},$$

$$V = \begin{bmatrix} v_1 & v_2 & v_3 \end{bmatrix} = \frac{1}{3\sqrt{5}}\begin{bmatrix} 5 & 0 & -2\sqrt{5} \\ 2 & 6 & \sqrt{5} \\ 4 & -3 & 2\sqrt{5} \end{bmatrix}.$$

You can easily check that $A = U\Sigma V^T$. □

Exercise 5.9.15 Write down the condensed SVD $A = \hat{U}\hat{\Sigma}\hat{V}^T$ (Theorem 4.1.10) of the matrix A in Example 5.9.14. □

Exercise 5.9.16 Let A be the matrix of Example 5.9.14. Calculate the eigenvalues and eigenvectors of A^TA. Compare them with the quantities calculated in Example 5.9.14. □

Exercise 5.9.17 Compute the SVD of the matrix $A = \begin{bmatrix} 3 & 4 \end{bmatrix}$. □

Exercise 5.9.18 Compute the SVD of the matrix

$$A = \begin{bmatrix} 3 & 2 \\ 2 & 3 \\ 2 & -2 \end{bmatrix}.$$

□

Exercise 5.9.19 (a) Compute the SVD of the matrix

$$A = \begin{bmatrix} 3 & 1 \\ 6 & 2 \end{bmatrix}.$$

(b) Compute the condensed SVD $A = \hat{U}\hat{\Sigma}\hat{V}^T$. □

Loss of Information through Squaring

Computation of the SVD by solving the eigenvalue problem for A^TA or AA^T is also a viable option for larger, more realistic problems, since we have efficient algorithms for solving the symmetric eigensystem problem. However, this approach has a serious disadvantage: If A has small but nonzero singular values, they will be calculated inaccurately. This is a consequence of the "loss of information through squaring" phenomenon (see Example 3.5.25), which occurs when we compute A^TA.

We can get some idea why this information loss occurs by considering an example. Suppose the entries of A are known to be correct to about six decimal places. If A has, say, $\sigma_1 \approx 1$ and $\sigma_{17} \approx 10^{-3}$, then σ_{17} is small compared with σ_1, but it is still well above the error level $\epsilon \approx 10^{-5}$ or 10^{-6}. We ought to be able to calculate σ_{17} with some precision, at least two or three decimal places. The entries of A^TA also have about six digits accuracy. Associated with the singular values σ_1 and σ_{17}, A^TA has eigenvalues $\lambda_1 = \sigma_1^2 \approx 1$ and $\lambda_{17} = \sigma_{17}^2 \approx 10^{-6}$. Since λ_{17} is of about the same magnitude as the errors in A^TA, we cannot expect to compute it with any accuracy at all.

Reduction to Bidiagonal Form

We now turn our attention to methods that calculate the SVD by operating directly on A. It turns out that many of the ideas that we developed for solving eigenvalue problems can also be applied to the SVD problem. For example, earlier in this chapter we found that the eigenvalue problem can be made much easier if we first reduce the matrix to a condensed form, such as tridiagonal or Hessenberg. The same is true of the SVD problem. The eigenvalue problem requires that the reduction be done via similarity transformations. For the singular value decomposition $A = U\Sigma V^T$, it is clear that similarity transformations are not required, but the transforming matrices should be orthogonal. Two matrices A, $B \in \mathbb{R}^{n\times m}$ are said to be *orthogonally equivalent* if there exist orthogonal matrices $P \in \mathbb{R}^{n\times n}$ and $Q \in \mathbb{R}^{m\times m}$ such that $B = PAQ$. We will see that we can reduce any matrix to bidiagonal form by an orthogonal equivalence transformation, in which each of the transforming matrices is a product of m or fewer reflectors. The algorithms that we are about to discuss are appropriate for dense matrices. We will not discuss the sparse SVD problem.

Exercise 5.9.20 Show that if two matrices are orthogonally equivalent, then they have the same singular values, and there are simple relationships between their singular vectors. □

We continue to assume that we are dealing with a matrix $A \in \mathbb{R}^{n\times m}$, but we will now make the additional assumption that $n \geq m$. This does not imply any loss of generality, for if $n < m$, we can operate on A^T instead of A. If the SVD of A^T is $A^T = U\Sigma V^T$, then the SVD of A is $A = V\Sigma^T U^T$.

A matrix $B \in \mathbb{R}^{n\times m}$ is said to be *bidiagonal* if $b_{ij} = 0$ whenever $i > j$ or $i < j - 1$. This means that B has the form

$$\begin{bmatrix} * & * & & & \\ & * & * & & \\ & & * & & \\ & & & \ddots & * \\ & & & & * \end{bmatrix}$$

with nonzero entries appearing only on two diagonals.

Theorem 5.9.21 *Let $A \in \mathbb{R}^{n\times m}$ with $n \geq m$. Then there exist orthogonal $\hat{U} \in \mathbb{R}^{n\times n}$ and $\hat{V} \in \mathbb{R}^{m\times n}$, both products of a finite number of reflectors, and a bidiagonal $B \in \mathbb{R}^{n\times m}$, such that*

$$A = \hat{U}B\hat{V}^T.$$

There is a finite algorithm to calculate $\hat{U}$, $\hat{V}$, and B.

Proof. We will prove Theorem 5.9.21 by describing the construction. It is quite similar to both the QR decomposition by reflectors (Section 3.2) and the reduction to upper Hessenberg form by reflectors (Section 5.5), so we will just sketch the procedure. The first step creates zeros in the first column and row of A. Let $\hat{U}_1 \in \mathbb{R}^{n\times n}$ be a reflector such that

$$\hat{U}_1 \begin{bmatrix} a_{11} \\ a_{21} \\ \vdots \\ a_{n1} \end{bmatrix} = \begin{bmatrix} * \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

Then the first column of $\hat{U}_1 A$ consists of zeros, except for the $(1,1)$ entry. Now let $\begin{bmatrix} \hat{a}_{11} & \hat{a}_{12} & \cdots & \hat{a}_{1m} \end{bmatrix}$ denote the first row of $\hat{U}_1 A$, and let $\check{V}_1$ be a reflector of the form

$$\hat{V}_1 = \left[\begin{array}{c|ccc} 1 & 0 & \cdots & 0 \\ \hline 0 & & & \\ \vdots & & \check{V}_1 & \\ 0 & & & \end{array}\right],$$

such that

$$\begin{bmatrix} \hat{a}_{12} & \cdots & \hat{a}_{1m} \end{bmatrix} \check{V}_1 = \begin{bmatrix} * & 0 & \cdots & 0 \end{bmatrix}.$$

Then the first row of $\hat{U}_1 A \hat{V}_1$ consists of zeros, except for the first two entries. Because the first column of $\hat{V}_1$ is e_1, the first column of $\hat{U}_1 A$ is unaltered by right multiplication

by $\hat{V}_1$. Thus $\hat{U}_1 A \hat{V}_1$ has the form

$$\hat{U}_1 A \hat{V}_1 = \left[\begin{array}{c|ccc} * & * & 0 & \cdots & 0 \\ \hline 0 & & & & \\ \vdots & & \hat{A} & & \\ 0 & & & & \end{array}\right].$$

The second step of the construction is identical to the first, except that it acts on the submatrix $\hat{A}$. It is easy to show that the reflectors used on the second step do not destroy the zeros created on the first step. After two steps we have

$$\hat{U}_2 \hat{U}_1 A \hat{V}_1 \hat{V}_2 = \left[\begin{array}{c|c|ccc} * & * & 0 & 0 & \cdots & 0 \\ \hline 0 & * & * & 0 & \cdots & 0 \\ \hline 0 & 0 & & & & \\ \vdots & \vdots & & \tilde{A} & & \\ 0 & 0 & & & & \end{array}\right].$$

The third step acts on the submatrix $\tilde{A}$, and so on. After m steps we have

$$\hat{U}_m \cdots \hat{U}_2 \hat{U}_1 A \hat{V}_1 \hat{V}_2 \cdots \hat{V}_{m-2} = \left[\begin{array}{ccccc} * & * & & & \\ & * & * & & \\ & & * & & \\ & & & \ddots & * \\ & & & & * \end{array}\right] = B.$$

Notice that steps $m-1$ and m require multiplications on the left only. Let $\hat{U} = \hat{U}_1 \hat{U}_2 \cdots \hat{U}_m$ and $\hat{V} = \hat{V}_1 \hat{V}_2 \cdots \hat{V}_{m-2}$. Then $\hat{U}^T A \hat{V} = B$; that is, $A = \hat{U} B \hat{V}^T$. □

Exercise 5.9.22 Carry out a flop count for this algorithm. Assume that $\hat{U}$ and $\hat{V}$ are not to be assembled explicitly. Recall from Section 3.2 that if $x \in \mathbb{R}^k$ and $U \in \mathbb{R}^{k \times k}$ is a reflector, then the operations $x \to Ux$ and $x^T \to x^T U$ each cost about $4k$ flops. Show that the cost of the right multiplications is slightly less than that of the left multiplications, but for large n and m, the difference is negligible. Notice that the total flop count is about twice that of a QR decomposition by reflectors. □

Exercise 5.9.23 Explain how the reflectors can be stored efficiently. How much storage space is required, in addition to the array that contains A initially? Assume that A is to be overwritten. □

In many applications (e.g. least squares problems) n is much larger than m. In this case it is sometimes more efficient to perform the reduction to bidiagonal form in two stages. In the first stage a QR decomposition of A is performed:

$$A = QR = \begin{bmatrix} Q_1 & Q_2 \end{bmatrix} \begin{bmatrix} \hat{R} \\ 0 \end{bmatrix},$$

where $\hat{R} \in \mathbb{R}^{m\times m}$ is upper triangular. This requires multiplications by reflectors on the left hand side of A only. In the second stage the relatively small matrix $\hat{R}$ is reduced to bidiagonal form $\hat{R} = \tilde{U}\tilde{B}\tilde{V}^T$. All of these matrices are $m \times m$. Then

$$A = \begin{bmatrix} Q_1 & Q_2 \end{bmatrix} \begin{bmatrix} \tilde{U} & 0 \\ 0 & I \end{bmatrix} \begin{bmatrix} \tilde{B} \\ 0 \end{bmatrix} \tilde{V}^T.$$

Letting

$$\hat{U} = \begin{bmatrix} Q_1 & Q_2 \end{bmatrix} \begin{bmatrix} \tilde{U} & 0 \\ 0 & I \end{bmatrix} = \begin{bmatrix} Q_1\tilde{U} & Q_2 \end{bmatrix} \in \mathbb{R}^{n\times n},$$

$$B = \begin{bmatrix} \tilde{B} \\ 0 \end{bmatrix} \in \mathbb{R}^{n\times m},$$

and $\hat{V} = \tilde{V} \in \mathbb{R}^{m\times m}$, we have $A = \hat{U}B\hat{V}^T$.

The advantage of this arrangement is that the right multiplications are applied to the small matrix $\hat{R}$ instead of the large matrix A. They therefore cost a lot less. The disadvantage is that the right multiplications destroy the upper-triangular form of $\hat{R}$. Thus most of the left multiplications must be repeated on the small matrix $\hat{R}$. If the ratio n/m is sufficiently large, the added cost of the left multiplications will be more than offset by the savings in the right multiplications. The break-even point is around $n/m = 5/3$. If $n/m \gg 1$, the flop count is cut nearly in half.

The various applications of the SVD have different requirements. Some require only the singular values, while others require the right or left singular vectors, or both. If any of the singular vectors are needed, then the matrices $\hat{U}$ and/or $\hat{V}$ have to be computed explicitly. Usually it is possible to avoid calculating $\hat{U}$, but there are numerous applications for which $\hat{V}$ is needed. The flop count of Exercise 5.9.22 does not include the cost of computing $\hat{U}$ or $\hat{V}$. If $\hat{U}$ is needed, then there is no point in doing the preliminary QR decomposition.

Computing the SVD of B

Since B is bidiagonal, it has the form

$$B = \begin{bmatrix} \tilde{B} \\ 0 \end{bmatrix},$$

where $\tilde{B} \in \mathbb{R}^{m\times m}$ is bidiagonal. The problem of finding the SVD of A is thus reduced to that of computing the SVD of the small bidiagonal matrix $\tilde{B}$.

For notational convenience we now drop the tilde from $\tilde{B}$ and let $B \in \mathbb{R}^{m\times m}$ be a bidiagonal matrix, say

$$B = \begin{bmatrix} \beta_1 & \gamma_1 & & & \\ & \beta_2 & \gamma_2 & & \\ & & \beta_3 & \ddots & \\ & & & \ddots & \gamma_{m-1} \\ & & & & \beta_m \end{bmatrix}. \tag{5.9.24}$$

Exercise 5.9.25 Show that both BB^T and B^TB are tridiagonal. □

The problem of computing the SVD of B is equivalent to that of finding the eigenvalues and eigenvectors of the symmetric, tridiagonal matrices B^TB and BB^T. There are numerous algorithms that can perform this task. One is the QR algorithm, which has commanded so much of our attention in this chapter. Others, including some really good ones, are discussed in Section 6.6. However, we prefer not to form the products B^TB and BB^T explicitly, for the same reason that we preferred not to form A^TA and AA^T. Thus the following question arises: Can the known methods for computing eigensystems of symmetric tridiagonal matrices be reformulated so that they operate directly on B, thus avoiding formation of the products? It turns out that the answer is yes for most (all?) of the methods. We will illustrate this by showing how to implement the QR algorithm on B^TB and BB^T without ever forming the products [32]. This is not necessarily the best algorithm for this problem, but it serves well as an illustration. It is closely related to the QZ algorithm for the generalized eigenvalue problem, which will be discussed in Section 6.7.

We begin with a definition. We will say that B is a *properly* bidiagonal matrix if (in the notation of (5.9.24)) $\beta_i \neq 0$ and $\gamma_i \neq 0$ for all i.

Exercise 5.9.26 Show that B is properly bidiagonal if and only if both BB^T and B^TB are properly tridiagonal matrices, that is, all of their off-diagonal entries are nonzero. □

If B is not properly bidiagonal, the problem of finding its SVD can be reduced to two or more smaller subproblems. First of all, if some γ_k is zero, then

$$B = \begin{bmatrix} B_1 & 0 \\ 0 & B_2 \end{bmatrix}, \tag{5.9.27}$$

where $B_1 \in \mathbb{R}^{k\times k}$ and $B_2 \in \mathbb{R}^{(m-k)\times(m-k)}$ are both bidiagonal. The SVDs of B_1 and B_2 can be found separately and then combined to yield the SVD of B. If some β_k is zero, a small amount of work transforms B to a form that can be reduced. The procedure is discussed in Exercise 5.9.46. If β_k is exactly zero, its removal is crucial to the functioning of the QR algorithm. However, if β_k is merely close to zero, it need not be removed; the QR algorithm will take care if it automatically.

We can now assume, without loss of generality, that B is a properly bidiagonal matrix. We can even assume (Exercise.5.9.47) that all of the β_k and γ_k are positive.

The QR Algorithm for the SVD

The basic facts about the QR algorithm, the fact that it is an iterative process, the benefits of shifting, and so forth, have been laid out in Section 5.6. Our main task here is to show how to do a QR iteration on B^TB or BB^T without forming these products.

We begin by writing down what the iteration would look like if we *were* willing to form the products. If we want to take a QR step on both B^TB and BB^T with a